

- No es posible conocer qué proceso se despertará, en caso de que haya alguno bloqueado, después de que se incremente el semáforo.
- Cuando se incrementa el semáforo, el número de procesos que se desbloquearán será uno o cero. Es decir, no se ha de despertar algún proceso necesariamente.

El uso de los semáforos como mecanismo de sincronización plantea una serie de **ventajas**, derivadas en parte de su simplicidad:

- Los semáforos permiten imponer restricciones explícitas con el objetivo de evitar errores por parte del programador.
- Las soluciones basadas en semáforos son, generalmente, limpias y organizadas, garantizando su simplicidad y permitiendo demostrar su validez.
- Los semáforos se pueden implementar de forma eficiente en una gran variedad de sistemas, fomentando así la portabilidad de las soluciones planteadas.

Antes de discutir los semáforos en POSIX, es importante destacar que la definición de semáforo discutida en esta sección requiere una **espera activa**. Mientras un proceso esté en su sección crítica, cualquier otro proceso que intente acceder a la suya deberá ejecutar de manera continuada un bucle en el código de entrada. Este esquema basado en espera activa hace que se desperdicien miles de ciclos de CPU en un sistema multiprogramado, aunque tiene la ventaja de que no se producen cambios de contexto.

Para evitar este tipo de problemática, es posible habilitar el bloqueo de un proceso que ejecuta *wait* sobre un semáforo con valor negativo. Esta operación de bloqueo coloca al proceso en una cola de espera vinculada con el semáforo de manera que el estado del proceso pasa a *en espera*. Posteriormente, el control pasa al planificador de la CPU, que seleccionará otro proceso para su ejecución.

Un proceso bloqueado se reanudará cuando otro proceso ejecuta *signal* sobre el semáforo, cambiando el estado del proceso original de *en espera* a *preparado*.

2.2. Implementación

2.2.1. Semáforos

En esta sección se discuten las **primitivas POSIX** más importantes relativas a la creación y manipulación de semáforos y segmentos de memoria compartida. Así mismo, se plantea el uso de una interfaz que facilite la manipulación de los semáforos mediante funciones básicas para la creación, destrucción e interacción mediante operaciones *wait* y *signal*.

En POSIX, existe la posibilidad de manejar **semáforos nombrados**, es decir, semáforos que se pueden manipular mediante cadenas de texto, facilitando así su manejo e incrementando el nivel semántico asociado a los mismos.

En el siguiente listado de código se muestra la interfaz de la primitiva `sem_open()`, utilizada para abrir un semáforo nombrado.

Listado 2.3: Primitiva `sem_open` en POSIX.

```
1 #include <semaphore.h>
2
3 /* Devuelve un puntero al semáforo o SEM_FAILED */
4 sem_t *sem_open (
5     const char *name, /* Nombre del semáforo */
6     int oflag,         /* Flags */
7     mode_t mode,       /* Permisos */
8     unsigned int value /* Valor inicial */
9 );
10
11 sem_t *sem_open (
12     const char *name, /* Nombre del semáforo */
13     int oflag,        /* Flags */
14 );
```

El valor de retorno de la primitiva `sem_open()` es un puntero a una estructura del tipo `sem_t` que se utilizará para llamar a otras funciones vinculadas al concepto de semáforo.

POSIX contempla diversas primitivas para cerrar (`sem_close()`) y eliminar (`sem_unlink()`) un semáforo, liberando así los recursos previamente creados por `sem_open()`. La primera tiene como parámetro un puntero a semáforo, mientras que la segunda sólo necesita el nombre del mismo.

Control de errores

Recuerde que primitivas como `sem_close` y `sem_unlink` también devuelven códigos de error en caso de fallo.

Listado 2.4: Primitivas `sem_close` y `sem_unlink` en POSIX.

```
1 #include <semaphore.h>
2
3 /* Devuelven 0 si todo correcto o -1 en caso de error */
4 int sem_close (sem_t *sem);
5 int sem_unlink (const char *name);
```

Finalmente, las primitivas para decrementar e incrementar un semáforo se exponen a continuación. Note cómo la primitiva de incremento utiliza el nombre *post* en lugar del tradicional *signal*.

Listado 2.5: Primitivas `sem_wait` y `sem_post` en POSIX.

```
1 #include <semaphore.h>
2
3 /* Devuelven 0 si todo correcto o -1 en caso de error */
4 int sem_wait (sem_t *sem);
5 int sem_post (sem_t *sem);
```

En el presente capítulo se utilizarán los semáforos POSIX para codificar soluciones a diversos problemas de sincronización que se irán

planteando en la sección 2.3. Para facilitar este proceso se ha definido una **interfaz** con el objetivo de simplificar la manipulación de dichos semáforos, el cual se muestra en el siguiente listado de código.

Listado 2.6: Interfaz de gestión de semáforos.

```

1 #include <semaphore.h>
2
3 typedef int bool;
4 #define false 0
5 #define true 1
6
7 /* Crea un semáforo POSIX */
8 sem_t *crear_sem (const char *name, unsigned int valor);
9
10 /* Obtener un semáforo POSIX (ya existente) */
11 sem_t *get_sem (const char *name);
12
13 /* Cierra un semáforo POSIX */
14 void destruir_sem (const char *name);
15
16 /* Incrementa el semáforo */
17 void signal_sem (sem_t *sem);
18
19 /* Decrementa el semáforo */
20 void wait_sem (sem_t *sem);

```

Patrón fachada

La interfaz de gestión de semáforos propuesta en esta sección es una implementación del patrón fachada *facade*.

El diseño de esta interfaz se basa en utilizar un esquema similar a los semáforos nombrados en POSIX, es decir, identificar a los semáforos por una cadena de caracteres. Dicha cadena se utilizará para crear un semáforo, obtener el puntero a la estructura de tipo *sem_t* y destruirlo. Sin embargo, las operaciones *wait* y *signal* se realizarán sobre el propio puntero a semáforo.

2.2.2. Memoria compartida

Los problemas de sincronización entre procesos suelen integrar algún tipo de recurso compartido cuyo acceso, precisamente, hace necesario el uso de algún mecanismo de sincronización como los semáforos. El recurso compartido puede ser un **fragmento de memoria**, por lo que resulta relevante estudiar los mecanismos proporcionados por el sistema operativo para dar soporte a la memoria compartida. En esta sección se discute el soporte POSIX para memoria compartida, haciendo especial hincapié en las primitivas proporcionadas y en un ejemplo particular.

La gestión de memoria compartida en POSIX implica la **apertura o creación** de un segmento de memoria compartida asociada a un nombre particular, mediante la primitiva *shm_open()*. En POSIX, los segmentos de memoria compartida actúan como archivos en memoria. Una vez abiertos, es necesario establecer el tamaño de los mismos mediante *ftruncate()* para, posteriormente, mapear el segmento al espacio de direcciones de memoria del usuario mediante la primitiva *mmap*.

A continuación se expondrán las primitivas POSIX necesarias para la gestión de segmentos de memoria compartida. Más adelante, se planteará un ejemplo concreto para compartir una variable entera.

Listado 2.7: Primitivas `shm_open` y `shm_unlink` en POSIX.

```
1 #include <mman.h>
2
3 /* Devuelve el descriptor de archivo o -1 si error */
4 int shm_open(
5     const char *name, /* Nombre del segmento */
6     int oflag,        /* Flags */
7     mode_t mode       /* Permisos */
8 );
9
10 /* Devuelve 0 si todo correcto o -1 en caso de error */
11 int shm_unlink(
12     const char *name /* Nombre del segmento */
13 );
```

Recuerde que, cuando haya terminado de usar el descriptor del archivo, es necesario utilizar `close()` como si se tratara de cualquier otro descriptor de archivo.

Listado 2.8: Primitiva `close`.

```
1 #include <unistd.h>
2
3 int close(
4     int fd /* Descriptor del archivo */
5 );
```

Después de crear un objeto de memoria compartida es necesario establecer de manera explícita su **longitud** mediante `ftruncate()`, ya que su longitud inicial es de cero bytes.

Listado 2.9: Primitiva `ftruncate`.

```
1 #include <unistd.h>
2 #include <sys/types.h>
3
4 int ftruncate(
5     int fd, /* Descriptor de archivo */
6     off_t length /* Nueva longitud */
7 );
```

A continuación, se **mapea** el objeto al espacio de direcciones del usuario mediante `mmap()`. El proceso inverso se realiza mediante la primitiva `munmap()`. La función `mmap()` se utiliza para la creación, consulta y modificación de valor almacena en el objeto de memoria compartida.

Listado 2.10: Primitiva mmap.

```
1 #include <sys/mman.h>
2
3 void *mmap(
4     void *addr,      /* Dirección de memoria */
5     size_t length,   /* Longitud del segmento */
6     int prot,        /* Protección */
7     int flags,       /* Flags */
8     int fd,          /* Descriptor de archivo */
9     off_t offset     /* Desplazamiento */
10 );
11
12 int munmap(
13     void *addr,      /* Dirección de memoria */
14     size_t length    /* Longitud del segmento */
15 );
```

Compartiendo un valor entero

En función del dominio del problema a resolver, el segmento u objeto de memoria compartida tendrá más o menos longitud. Por ejemplo, sería posible definir una estructura específica para un **tipo abstracto de datos** y crear un segmento de memoria compartida para que distintos procesos puedan modificar dicho segmento de manera concurrente. En otros problemas sólo será necesario utilizar un valor entero a compartir entre los procesos. La creación de un segmento de memoria para este último caso se estudiará a continuación.

Al igual que en el caso de los semáforos, se ha definido una **interfaz** para simplificar la manipulación de un valor entero compartido. Dicha interfaz facilita la creación, destrucción, modificación y consulta de un valor entero que puede ser compartido por múltiples procesos. Al igual que en el caso de los semáforos, la manipulación del valor entero se realiza mediante un esquema basado en nombres, es decir, se hacen uso de cadenas de caracteres para identificar los segmentos de memoria compartida.

Listado 2.11: Interfaz de gestión de un valor entero compartido.

```
1 /* Crea un objeto de memoria compartida */
2 /* Devuelve el descriptor de archivo */
3 int crear_var (const char *name, int valor);
4
5 /* Obtiene el descriptor asociado a la variable */
6 int obtener_var (const char *name);
7
8 /* Destruye el objeto de memoria compartida */
9 void destruir_var (const char *name);
10
11 /* Modifica el valor del objeto de memoria compartida */
12 void modificar_var (int shm_fd, int valor);
13
14 /* Devuelve el valor del objeto de memoria compartida */
15 void consultar_var (int shm_fd, int *valor);
```

Resulta interesante resaltar que la función *consultar* almacena el valor de la consulta en la variable *valor*, pasada por referencia. Este esquema es más eficiente, evita la generación de copias en la propia función y facilita el proceso de *unmapping*.

En el siguiente listado de código se muestra la implementación de la función *crear_var* cuyo objetivo es el de crear e inicializar un segmento de memoria compartida para un valor entero. Note cómo es necesario llevar a cabo el *mapping* del objeto de memoria compartida con el objetivo de asignarle un valor (en este caso un valor entero).

Listado 2.12: Creación de un segmento de memoria compartida.

```
1 int crear_var (const char *name, int valor) {
2     int shm_fd;
3     int *p;
4
5     /* Abre el objeto de memoria compartida */
6     shm_fd = shm_open(name, O_CREAT | O_RDWR, 0644);
7     if (shm_fd == -1) {
8         fprintf(stderr, "Error al crear la variable: %s\n",
9             strerror(errno));
10        exit(EXIT_FAILURE);
11    }
12
13    /* Establecer el tamaño */
14    if (ftruncate(shm_fd, sizeof(int)) == -1) {
15        fprintf(stderr, "Error al trincar la variable: %s\n",
16            strerror(errno));
17        exit(EXIT_FAILURE);
18    }
19
20    /* Mapeo del objeto de memoria compartida */
21    p = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE, MAP_SHARED,
22        shm_fd, 0);
23    if (p == MAP_FAILED) {
24        fprintf(stderr, "Error al mapear la variable: %s\n",
25            strerror(errno));
26        exit(EXIT_FAILURE);
27    }
28    *p = valor;
29    munmap(p, sizeof(int));
30
31    return shm_fd;
32 }
```

Posteriormente, es necesario establecer el tamaño de la variable compartida, mediante la función *ftruncate*, asignar de manera explícita el valor y, finalmente, realizar el proceso de *unmapping* mediante la función *munmap*.

2.3. Problemas clásicos de sincronización

En esta sección se plantean algunos problemas clásicos de sincronización y se discuten posibles soluciones basadas en el uso de semáforos. En la mayor parte de ellos también se hace uso de segmentos de memoria compartida para compartir datos.

2.3.1. El buffer limitado

El problema del buffer limitado ya se presentó en la sección 1.2.1 y también se suele denominar el *problema del productor/consumidor*. Básicamente, existe un espacio de almacenamiento común limitado, es decir, dicho espacio consta de un conjunto finito de huecos que pueden contener o no elementos. Por una parte, los productores insertarán elementos en el buffer. Por otra parte, los consumidores los extraerán.

La primera cuestión a considerar es que se ha de controlar el acceso exclusivo a la **sección crítica**, es decir, al propio buffer. La segunda cuestión importante reside en controlar **dos situaciones**: i) no se puede insertar un elemento cuando el buffer está lleno y ii) no se puede extraer un elemento cuando el buffer está vacío. En estos dos casos, será necesario establecer un mecanismo para bloquear a los procesos correspondientes.

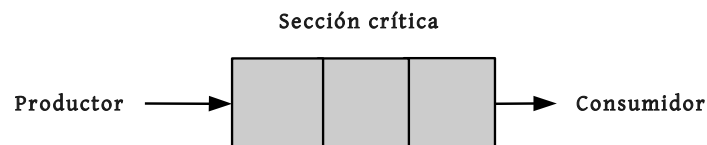


Figura 2.4: Esquema gráfico del problema del buffer limitado.

Para controlar el acceso exclusivo a la sección crítica se puede utilizar un semáforo binario, de manera que la primitiva *wait* posibilite dicho acceso o bloquee a un proceso que desee acceder a la sección crítica. Para controlar las dos situaciones anteriormente mencionadas, se pueden usar dos semáforos independientes que se actualizarán cuando se produzca o se consuma un elemento del buffer, respectivamente.

La solución planteada se basa en los siguientes elementos:

- **mutex**, semáforo binario que se utiliza para proporcionar exclusión mutua para el acceso al buffer de productos. Este semáforo se inicializa a 1.
- **empty**, semáforo contador que se utiliza para controlar el número de huecos vacíos del buffer. Este semáforo se inicializa a n , siendo n el tamaño del buffer.

```
while (1) {  
    // Produce en nextP.  
    wait (empty);  
    wait (mutex);  
    // Guarda nextP en buffer.  
    signal (mutex);  
    signal (full);  
}
```

```
while (1) {  
    wait (full);  
    wait (mutex);  
    // Rellenar nextC.  
    signal (mutex);  
    signal (empty);  
    // Consume nextC.  
}
```

Figura 2.5: Procesos productor y consumidor sincronizados mediante un semáforo.

- **full**, semáforo contador que se utiliza para controlar el número de huecos llenos del buffer. Este semáforo se inicializa a 0.

En esencia, los semáforos *empty* y *full* garantizan que no se puedan insertar más elementos cuando el buffer esté lleno o extraer más elementos cuando esté vacío, respectivamente. Por ejemplo, si el valor interno de *empty* es 0, entonces un proceso que ejecute *wait* sobre este semáforo se quedará bloqueado hasta que otro proceso ejecute *signal*, es decir, hasta que otro proceso produzca un nuevo elemento en el buffer.

La figura 2.5 muestra una posible solución, en pseudocódigo, del problema del buffer limitado. Note cómo el proceso *productor* ha de esperar a que haya algún hueco libre antes de producir un nuevo elemento, mediante la operación *wait(empty)*. El acceso al buffer se controla mediante *wait(mutex)*, mientras que la liberación se realiza con *signal(mutex)*. Finalmente, el *productor* indica que hay un nuevo elemento mediante *signal(empty)*.

Patrones de sincronización

En la solución planteada para el problema del buffer limitado se pueden extraer dos patrones básicos de sincronización con el objetivo de reusarlos en problemas similares.

Uno de los patrones utilizados es el **patrón mutex** [5], que básicamente permite controlar el acceso concurrente a una variable compartida para que éste sea exclusivo. En este caso, dicha variable compartida es el buffer de elementos. El *mutex* se puede entender como la llave que pasa de un proceso a otro para que este último pueda proceder. En otras palabras, un proceso (o hilo) ha de tener acceso al *mutex* para poder acceder a la variable compartida. Cuando termine

de trabajar con la variable compartida, entonces el proceso liberará el *mutex*.

El patrón *mutex* se puede implementar de manera sencilla mediante un semáforo binario. Cuando un proceso intente adquirir el *mutex*, entonces ejecutará *wait* sobre el mismo. Por el contrario, deberá ejecutar *signal* para liberarlo.

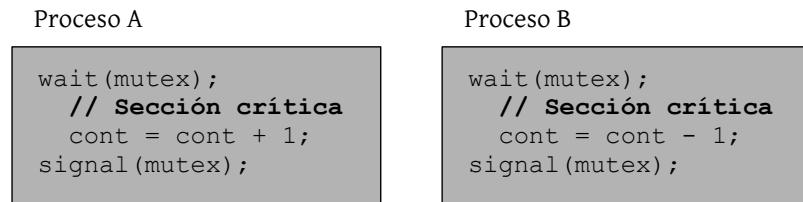


Figura 2.6: Aplicación del patrón *mutex* para acceder a la variable compartida *cont*.

Por otra parte, en la solución del problema anterior también se ve reflejado un patrón de sincronización que está muy relacionado con uno de los principales usos de los semáforos: el **patrón señalización** [5]. Básicamente, este patrón se utiliza para que un proceso o hilo pueda notificar a otro que un determinado evento ha ocurrido. En otras palabras, el patrón *señalización* garantiza que una sección de código de un proceso se ejecute antes que otra sección de código de otro proceso, es decir, resuelve el problema de la **serialización**.

En el problema del buffer limitado, este patrón lo utiliza el consumidor para notificar que existe un nuevo *ítem* en el buffer, posibilitando así que el productor pueda obtenerlo, mediante el *wait* correspondiente sobre el semáforo *empty*.

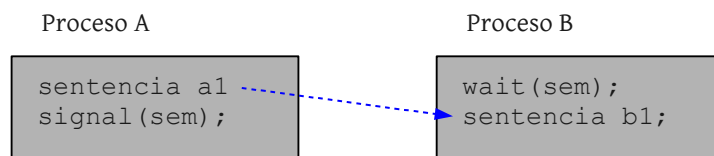


Figura 2.7: Patrón *señalización* para la serialización de eventos.

2.3.2. Lectores y escritores

Suponga una estructura o una base de datos compartida por varios procesos concurrentes. Algunos de estos procesos simplemente tendrán que leer información, mientras que otros tendrán que actualizarla, es decir, realizar operaciones de lectura y escritura. Los primeros procesos se denominan lectores mientras que el resto serán escritores.

Si dos o más lectores intentan acceder de manera concurrente a los datos, entonces no se generará ningún problema. Sin embargo, un acceso simultáneo de un escritor y cualquier otro proceso, ya sea lector o escritor, generaría una **condición de carrera**.

El problema de los lectores y escritores se ha utilizado en innumerables ocasiones para probar distintas primitivas de sincronización y existen multitud de variaciones sobre el problema original. La **versión más simple** se basa en que ningún lector tendrá que esperar a menos que ya haya un escritor en la sección crítica. Es decir, ningún lector ha de esperar a que otros lectores terminen porque un proceso escritor esté esperando.

En otras palabras, un escritor no puede acceder a la sección crítica si alguien, ya sea lector o escritor, se encuentra en dicha sección. Por otra parte, mientras un escritor se encuentra en la sección crítica, ninguna otra entidad puede acceder a la misma. Desde otro punto de vista, mientras haya lectores, los escritores esperarán y, mientras haya un escritor, los lectores no leerán.

El número de lectores representa una variable compartida, cuyo acceso se deberá controlar mediante un semáforo binario aplicando, de nuevo, el patrón *mutex*. Por otra parte, es necesario modelar la sincronización existente entre los lectores y los escritores, es decir, habrá que considerar el tipo de proceso que quiere acceder a la sección crítica, junto con el estado actual, para determinar si un proceso puede acceder o no a la misma. Para ello, se puede utilizar un semáforo binario que gobierne el acceso a la sección crítica en función del tipo de proceso (escritor o lector), considerando la **prioridad de los lectores** frente a los escritores.

La solución planteada, cuyo pseudocódigo se muestra en la figura 2.8, hace uso de los siguientes elementos:

- **num_lectores**, una variable compartida, inicializada a 0, que sirve para contar el número de lectores que acceden a la sección crítica.
- **mutex**, un semáforo binario, inicializado a 1, que controla el acceso concurrente a *num_lectores*.
- **acceso_esc**, un semáforo binario, inicializado a 1, que sirve para dar paso a los escritores.

En esta primera versión del problema se prioriza el acceso de los lectores con respecto a los escritores, es decir, si un lector entra en la sección crítica, entonces se dará prioridad a otros lectores que estén esperando frente a un escritor. La implementación garantiza esta prioridad debido a la última sentencia condicional del código del proceso lector, ya que sólo ejecutará *signal* sobre el semáforo *acceso_esc* cuando no haya lectores.

Proceso lector

```

wait(mutex);
  num_lectores++;
  if num_lectores == 1 then
    wait(acceso_esc);
signal(mutex);
/* SC: Lectura */
/* ... */
wait(mutex);
  num_lectores--;
  if num_lectores == 0 then
    signal(acceso_esc);
signal(mutex);

```

Proceso escritor

```

wait(acceso_esc);
/* SC: Escritura */
/* ... */
signal(acceso_esc);

```

.....

Figura 2.8: Pseudocódigo de los procesos lector y escritor.

Cuando un proceso se queda esperando una gran cantidad de tiempo porque otros procesos tienen prioridad en el uso de los recursos, éste se encuentra en una situación de inanición o *starvation*.

La solución planteada garantiza que no existe posibilidad de interbloqueo pero se da una situación igualmente peligrosa, ya que un escritor se puede quedar esperando indefinidamente en un estado de inanición (*starvation*) si no dejan de pasar lectores.

Priorizando escritores

En esta sección se plantea una modificación sobre la solución anterior con el objetivo de que, cuando un escritor llegue, los lectores puedan finalizar pero no sea posible que otro lector tenga prioridad sobre el escritor.

Para modelar esta variación sobre el problema original, será necesario contemplar de algún modo esta nueva prioridad de un escritor sobre los lectores que estén esperando para acceder a la sección crítica. En otras palabras, será necesario integrar algún tipo de mecanismo que priorice el **turno** de un escritor frente a un lector cuando ambos se encuentren en la sección de entrada.

Una posible solución consiste en añadir un semáforo *turno* que gobierne el acceso de los lectores de manera que los escritores puedan adquirirlo de manera previa. En la imagen 2.9 se muestra el pseudocódigo de los procesos lector y escritor, modificados a partir de la versión original.



¿Cómo modificaría la solución propuesta para garantizar que, cuando un escritor llegue, ningún lector pueda entrar en su sección crítica hasta que todos los escritores hayan terminado?

Patrones de sincronización

El problema de los lectores y escritores se caracteriza porque la presencia de un proceso en la sección crítica no excluye necesariamente a otros procesos. Por ejemplo, la existencia de un proceso lector en la sección crítica posibilita que otro proceso lector acceda a la misma. No obstante, note cómo el acceso a la variable compartida *num_lectores* se realiza de manera exclusiva mediante un *mutex*.

Sin embargo, la presencia de una categoría, por ejemplo la categoría *escritor*, en la sección crítica sí que excluye el acceso de procesos de otro tipo de categoría, como ocurre con los lectores. Este patrón de exclusión se suele denominar **exclusión mutua categórica** [5].

Por otra parte, en este problema también se da una situación que se puede reproducir con facilidad en otros problemas de sincronización. Básicamente, esta situación consiste en que el primer proceso en acceder a una sección de código adquiera el semáforo, mientras que el último lo libera. Note cómo esta situación ocurre en el proceso lector.

El nombre del patrón asociado a este tipo de situaciones es **light-switch** o **interruptor** [5], debido a su similitud con un interruptor de luz (la primera persona lo activa o enciende mientras que la última lo desactiva o apaga).

Listado 2.13: Implementación del patrón interruptor.

```
1 import threading
2 from threading import Semaphore
3
4 class Interruptor:
5
6     def __init__(self):
7         self._contador = 0
8         self._mutex = Semaphore(1)
9
10    def activar (self, semaforo):
11        self._mutex.acquire()
12        self._contador += 1
13        if self._contador == 1:
14            semaforo.acquire()
15        self._mutex.release()
16
17    def desactivar (self, semaforo):
18        self._mutex.acquire()
19        self._contador -= 1
20        if self._contador == 0:
21            semaforo.release()
22        self._mutex.release()
```

En el caso de utilizar un enfoque orientado a objetos, este patrón se puede encapsular fácilmente mediante una clase, como se muestra en el siguiente listado de código.

Finalmente, el uso del semáforo *turno* está asociado a patrón **barrera** [5] debido a que la adquisición de *turno* por parte de un escritor crea una barrera para el resto de lectores, bloqueando el acceso a la sección crítica hasta que el escritor libere el semáforo mediante *signal*, es decir, hasta que levante la barrera.

2.3.3. Los filósofos comensales

Los filósofos se encuentran comiendo o pensando. Todos ellos comparten una mesa redonda con cinco sillas, una para cada filósofo. En el centro de la mesa hay una fuente de arroz y en la mesa sólo hay cinco palillos, de manera que cada filósofo tiene un palillo a su izquierda y otro a su derecha.

Cuando un filósofo piensa, entonces se abstrae del mundo y no se relaciona con otros filósofos. Cuando tiene hambre, entonces intenta acceder a los palillos que tiene a su izquierda y a su derecha (necesita ambos). Naturalmente, un filósofo no puede quitarle un palillo a otro filósofo y sólo puede comer cuando ha cogido los dos palillos. Cuando un filósofo termina de comer, deja los palillos y se pone a pensar.

Este problema es uno de los **problemas clásicos de sincronización** entre procesos, donde es necesario gestionar el acceso concurrente a los recursos compartidos, es decir, a los propios palillos. En este contexto, un palillo se puede entender como un recurso indivisible, es decir, en todo momento se encontrará en un estado de uso o en un estado de no uso.

La solución planteada a continuación se basa en representar cada palillo con un semáforo inicializado a 1. De este modo, un filósofo que intente hacerse con un palillo tendrá que ejecutar *wait* sobre el mismo, mientras que ejecutará *signal* para liberarlo (ver figura 2.11). Recuerde que un filósofo sólo puede acceder a los palillos adyacentes al mismo.

Aunque la solución planteada garantiza que dos filósofos que estén sentados uno al lado del otro nunca coman juntos, es posible que el sistema alcance una situación de interbloqueo. Por ejemplo, suponga que todos los filósofos cogen el palillo situado a su izquierda. En ese momento, todos los semáforos asociados estarán a 0, por lo que ningún filósofo podrá coger el palillo situado a su derecha.

Evitando el interbloqueo

Una de las modificaciones más directas para evitar un interbloqueo en la versión clásica del problema de los filósofos comensales es limitar el **número de filósofos a cuatro**. De este modo, si sólo hay cuatro filósofos, entonces en el peor caso todos cogerán un palillo pero siempre quedará un palillo libre. En otras palabras, si dos filósofos vecinos

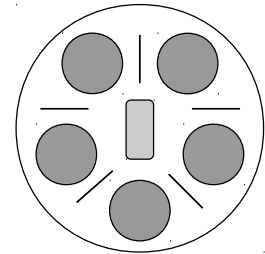


Figura 2.10: Representación gráfica del problema de los filósofos comensales. En este caso concreto, cinco filósofos comparten cinco palillos. Para poder comer, un filósofo necesita obtener dos palillos: el de su izquierda y el de su derecha.

Proceso filósofo

```

while (1) {
    // Pensar...
    wait(palillos[i]);
    wait(palillos[(i+1) % 5]);
    // Comer...
    signal(palillos[i]);
    signal(palillos[(i+1) % 5]);
}

```

Figura 2.11: Pseudocódigo de la solución del problema de los filósofos comensales con riesgo de interbloqueo.

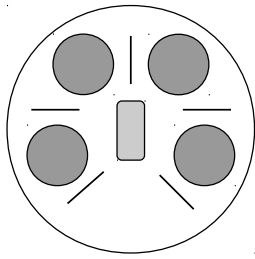


Figura 2.12: Problema de los filósofos con cuatro comensales en la mesa.

ya tienen un palillo, entonces al menos uno de ellos podrá utilizar el palillo disponible para empezar a comer.

El número de filósofos de la mesa se puede controlar con un semáforo contador, inicializado a dicho número. A continuación se muestra el pseudocódigo de una posible solución a esta variante del problema de los filósofos comensales. Como se puede apreciar, el semáforo *sirviente* gobierna el acceso a los palillos.

Proceso filósofo

```

while (1) {
    /* Pensar... */
    wait(sirviente);
    wait(palillos[i]);
    wait(palillos[(i+1)]);
    /* Comer... */
    signal(palillos[i]);
    signal(palillos[(i+1)]);
    signal(sirviente);
}

```

Figura 2.13: Pseudocódigo de la solución del problema de los filósofos comensales sin riesgo de interbloqueo.

Además de evitar el *deadlock*, esta solución garantiza que ningún filósofo se muera de hambre. Para ello, imagine la situación en la que los dos vecinos de un filósofo están comiendo, de manera que este último está bloqueado por ambos. Eventualmente, uno de los dos vecinos dejará su palillo. Debido a que el filósofo que se encontraba bloqueado era el único que estaba esperando a que el palillo estuviera libre, entonces lo cogerá. Del mismo modo, el otro palillo quedará disponible en algún instante.

Proceso filósofo i

```

#define N 5
#define IZQ (i-1) % 5
#define DER (i+1) % 5

int estado[N];
sem_t mutex;
sem_t sem[N];

void filosofo (int i) {
    while (1) {
        pensar();
        coger_palillos(i);
        comer();
        dejar_palillos(i);
    }
}

void coger_palillos (int i) {
    wait(mutex);
    estado[i] = HAMBRIENTO;
    prueba(i);
    signal(mutex);
    wait(sem[i]);
}

void dejar_palillos (int i) {
    wait(mutex);
    estado[i] = PENSANDO;
    prueba(i - 1);
    prueba(i + 1);
    signal(mutex);
}

void prueba (int i) {
    if (estado[i] == HAMBRIENTO &&
        estado[izq] != COMIENDO &&
        estado[der] != COMIENDO) {
        estado[i] = COMIENDO;
        signal(sem[i]);
    }
}

```

Figura 2.14: Pseudocódigo de la solución del problema de los filósofos comensales sin interbloqueo.

Otra posible modificación, propuesta en el libro de A.S. Tanenbaum *Sistemas Operativos Modernos* [15], para evitar la posibilidad de interbloqueo consiste en permitir que un filósofo coja sus palillos si y sólo si **ambos palillos están disponibles**. Para ello, un filósofo deberá obtener los palillos dentro de una sección crítica. En la figura 2.14 se muestra el pseudocódigo de esta posible modificación, la cual incorpora un array de enteros, denominado *estado* compartido entre los filósofos, que refleja el estado de cada uno de ellos. El acceso a dicho array ha de protegerse mediante un semáforo binario, denominado *mutex*.

La principal diferencia con respecto al planteamiento anterior es que el array de semáforos actual sirve para gestionar el estado de los filósofos, en lugar de gestionar los palillos. Note cómo la función *coger_palillos()* comprueba si el filósofo *i* puede coger sus dos palillos adyacentes mediante la función *prueba()*.

La función *dejar_palillos()* está planteada para que el filósofo *i* compruebe si los filósofos adyacentes están hambrientos y no se encuentran comiendo.

Note que tanto para acceder al estado de un filósofo como para invocar a *prueba()*, un proceso ha de adquirir el *mutex*. De este modo, la operación de comprobar y actualizar el array se transforma en **atómica**. Igualmente, no es posible entrar en una situación de interbloqueo debido a que el único semáforo accedido por más de un filósofo es *mutex* y ningún filósofo ejecuta *wait* con el semáforo adquirido.

Compartiendo el arroz

Una posible variación del problema original de los filósofos comensales consiste en suponer que los filósofos comerán directamente de

una **bandeja situada en el centro** de la mesa. Para ello, los filósofos disponen de cuatro palillos que están dispuestos al lado de la bandeja. Así, cuando un filósofo quiere comer, entonces tendrá que coger un par de palillos, comer y, posteriormente, dejarlos de nuevo para que otro filósofo pueda comerlos. La figura 2.15 muestra de manera gráfica la problemática planteada.

Esta variación admite varias soluciones. Por ejemplo, se podría utilizar un esquema similar al discutido en la solución anterior en la que se evitaba el interbloqueo. Es decir, se podría pensar una solución en la que se comprobara, de manera explícita, si un filósofo puede coger dos palillos, en lugar de coger uno y luego otro, antes de comer.

Sin embargo, es posible plantear una solución más simple y elegante ante esta variación. Para ello, se puede utilizar un **semáforo contador** inicializado a 2, el cual representa los pares de palillos disponibles para los filósofos. Este semáforo *palillos* gestiona el acceso concurrente de los filósofos a la bandeja de arroz.

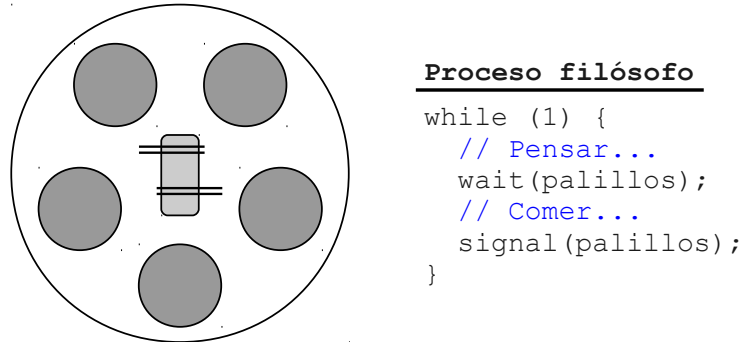


Figura 2.15: Variación con una bandeja al centro del problema de los filósofos comensales. A la izquierda se muestra una representación gráfica, mientras que a la derecha se muestra una posible solución en pseudocódigo haciendo uso de un semáforo contador.

Patrones de sincronización

La primera solución sin interbloqueo del problema de los filósofos comensales y la solución a la variación con los palillos en el centro comparten una característica. En ambas soluciones se ha utilizado un semáforo contador que posibilita el acceso concurrente de varios procesos a la sección crítica.

En el primer caso, el semáforo *serviente* se inicializaba a 4 para permitir el acceso concurrente de los filósofos a la sección crítica, aunque luego compitieran por la obtención de los palillos a nivel individual.

En el segundo caso, el semáforo *palillos* se inicializaba a 2 para permitir el acceso concurrente de dos procesos *filósofo*, debido a que los filósofos disponían de 2 pares de palillos.

Este tipo de soluciones se pueden encuadrar dentro de la aplicación del patrón **multiplex** [5], mediante el uso de un semáforo contador cu-

ya inicialización determina el número máximo de procesos que pueden acceder de manera simultánea a una sección de código. En cualquier otro momento, el valor del semáforo representa el número de procesos que pueden entrar en dicha sección.

Si el semáforo alcanza el valor 0, entonces el siguiente proceso que ejecute *wait* se bloqueará. Por el contrario, si todos los procesos abandonan la sección crítica, entonces el valor del semáforo volverá a n .

```
wait(multiplex);  
    // Sección crítica  
signal(multiplex);
```

Figura 2.16: Patrón *multiplex* para el acceso concurrente de múltiples procesos.

2.3.4. El puente de un solo carril

Suponga un puente que tiene una carretera con un único carril por el que los coches pueden circular en un sentido o en otro. La anchura del carril hace imposible que dos coches puedan pasar de manera simultánea por el puente. El protocolo utilizado para atravesar el puente es el siguiente:

- Si no hay ningún coche circulando por el puente, entonces el primer coche en llegar cruzará el puente.
- Si un coche está atravesando el puente de norte a sur, entonces los coches que estén en el extremo norte del puente tendrán prioridad sobre los que vayan a cruzarlo desde el extremo sur.
- Del mismo modo, si un coche se encuentra cruzando de sur a norte, entonces los coches del extremo sur tendrán prioridad sobre los del norte.

En este problema, el puente se puede entender como el recurso que comparten los coches de uno y otro extremo, el cual les permite continuar con su funcionamiento habitual. Sin embargo, los coches se han de sincronizar para acceder a dicho puente.

Por otra parte, en la solución se ha de contemplar la cuestión de la prioridad, es decir, es necesario tener en cuenta que si, por ejemplo, mientras un coche del norte esté en el puente, entonces si vienen más coches del norte podrán circular por el puente. Es decir, un coche que *adquiere* el puente habilita al resto de coches del norte a la hora de cruzarlo, manteniendo la preferencia con respecto a los coches que, posiblemente, estén esperando en el sur.

Por lo tanto, es necesario controlar, de algún modo, el número de coches en circulación desde ambos extremos. Si ese número llega a 0, entonces los coches del otro extremo podrán empezar a circular.

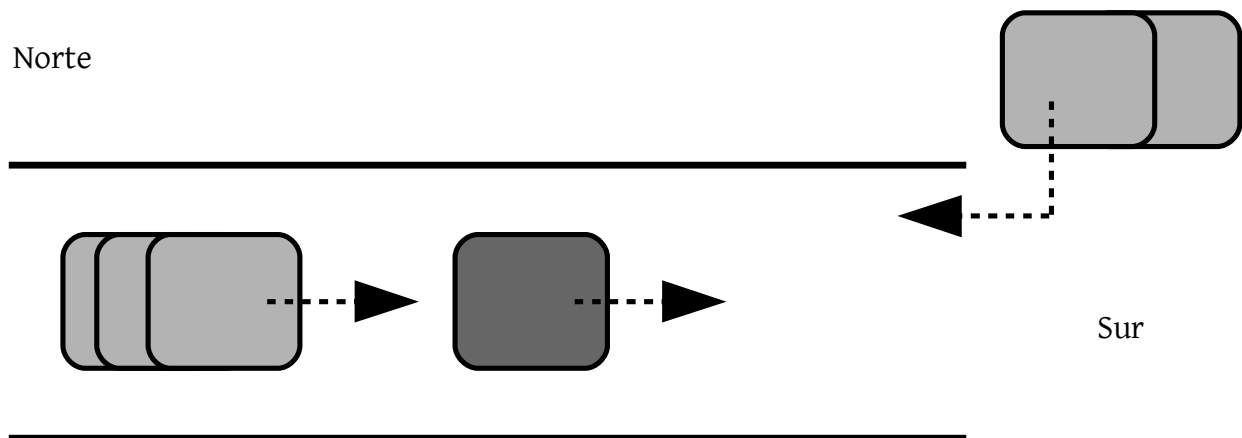


Figura 2.17: Esquema gráfico del problema del puente de un solo carril.

Una posible solución manejaría los siguientes elementos:

- **Puente**, un semáforo binario que gestiona el acceso al mismo.
- **CochesNorte**, una variable entera compartida para almacenar el número de coches que circulan por el puente, en un determinado instante del tiempo, desde el extremo norte.
- **MutexN**, un semáforo binario que permite controlar el acceso exclusivo a *CochesNorte*.
- **CochesSur**, una variable entera compartida para almacenar el número de coches que circulan por el puente, en un determinado instante del tiempo, desde el extremo sur.
- **MutexS**, un semáforo binario que permite controlar el acceso exclusivo a *CochesSur*.

En el siguiente listado se muestra el pseudocódigo de una posible solución al problema del puente de un solo carril. Básicamente, si un coche viene del extremo norte, y es el primero, entonces intenta acceder al puente (líneas **2-4**) invocando a *wait* sobre el semáforo *puente*.

Si el puente no estaba ocupado, entonces podrá cruzarlo, decrementando el semáforo *puente* a 0 y bloqueando a los coches del sur. Cuando lo haya cruzado, decrementará el contador de coches que vienen del extremo norte. Para ello, tendrá que usar el semáforo binario *mutexN*. Si la variable *cochesNorte* llega a 0, entonces liberará el puente mediante *signal* (línea **11**).

Este planteamiento se aplica del mismo modo a los coches que provienen del extremo sur, cambiando los semáforos y la variable compartida.

Listado 2.14: Proceso CocheNorte.

```
1 wait(mutexN);
2 cochesNorte++;
3 if cochesNorte == 1 then
4     wait(puente);
5 signal(mutexN);
6 /* SC: Cruzar el puente */
7 /* ... */
8 wait(mutexN);
9 cochesNorte--;
10 if cochesNorte == 0 then
11     signal(puente);
12 signal(mutexN);
```

El anexo A contiene una solución completa de este problema de sincronización. La implementación de *coche* es genérica, es decir, independiente de si parte del extremo norte o del extremo sur. La información necesaria para su correcto funcionamiento (básicamente el nombre de los semáforos y las variables de memoria compartida) se envía desde el proceso *manager* (el que ejecuta la primitiva *excl()*).

2.3.5. El barbero dormilón

El problema original de la barbería fue propuesto por Dijkstra, aunque comúnmente se suele conocer como el problema del barbero dormilón. El enunciado de este problema clásico de sincronización se expone a continuación.

La barbería tiene una sala de espera con n sillas y la habitación del barbero tiene una única silla en la que un cliente se sienta para que el barbero trabaje. Si no hay clientes, entonces el barbero se duerme. Si un cliente entra en la barbería y todas las sillas están ocupadas, es decir, tanto la del barbero como las de la sala de espera, entonces el cliente se marcha. Si el barbero está ocupado pero hay sillas disponibles, entonces el cliente se sienta en una de ellas. Si el barbero está durmiendo, entonces el cliente lo despierta.

Para abordar este problema clásico se describirán algunos pasos que pueden resultar útiles a la hora de enfrentarse a un problema de sincronización entre procesos. En cada uno de estos pasos se discutirá la problemática particular del problema de la barbería.

1. **Identificación de procesos**, con el objetivo de distinguir las distintas entidades que forman parte del problema. En el caso del barbero dormilón, estos procesos son el *barbero* y los *clientes*. Típicamente, será necesario otro proceso *manager* encargado del lanzamiento y gestión de los procesos específicos de dominio.
2. **Agrupación en clases** o trozos de código, con el objetivo de estructurar a nivel de código la solución planteada. En el caso del barbero, el código se puede estructurar de acuerdo a la identificación de procesos.
3. **Identificación de recursos compartidos**, es decir, identificación de aquellos elementos compartidos por los procesos. Será nece-

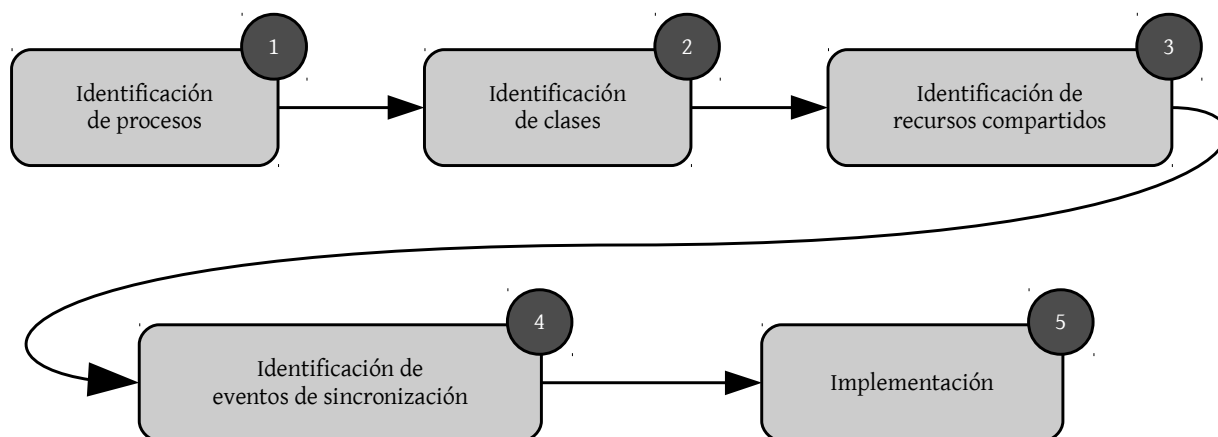


Figura 2.18: Secuencia de pasos para diseñar una solución ante un problema de sincronización.

sario asociar herramientas de sincronización para garantizar el correcto acceso a dichos recursos. Por ejemplo, se tendrá que usar un semáforo para controlar el acceso concurrente al número de sillas.

4. **Identificación de eventos de sincronización**, con el objetivo de delimitar qué eventos tienen que ocurrir obligatoriamente antes que otros. Por ejemplo, antes de poder actuar, algún cliente ha de despertar al barbero. Será necesario asociar herramientas de sincronización para controlar dicha sincronización. Por ejemplo, se podría pensar en un semáforo binario para que un cliente despierte al barbero.
5. **Implementación**, es decir, codificación de la solución planteada utilizando algún lenguaje de programación. Por ejemplo, una alternativa es el uso de los mecanismos que el estándar POSIX define para la manipulación de mecanismos de sincronización y de segmentos de memoria compartida.

En el caso particular del problema del barbero dormilón se distinguen claramente dos **recursos compartidos**:

- **Las sillas**, compartidas por los clientes en la sala de espera.
- **El sillón**, compartido por los clientes que compiten para ser atendidos por el barbero.

Para modelar las sillas se ha optado por utilizar un **segmento de memoria compartida** que representa una variable entera. Con dicha variable se puede controlar fácilmente el número de clientes que se encuentran en la barbería, modelando la situación específica en la que todas las sillas están ocupadas y, por lo tanto, el cliente ha de marcharse. La variable denominada *num_clientes* se inicializa a 0. El

acceso a esta variable ha de ser exclusivo, por lo que una vez se utiliza el patrón *mutex*.

Otra posible opción hubiera sido un semáforo contador inicializado al número inicial de sillas libres en la sala de espera de la barbería. Sin embargo, si utilizamos un semáforo no es posible modelar la restricción que hace que un cliente abandone la barbería si no hay sillas libres. Con un semáforo, el cliente se quedaría bloqueado hasta que hubiese un nuevo hueco. Por otra parte, para modelar el recurso compartido representado por el sillón del barbero se ha optado por un **semáforo binario**, denominado *sillón*, con el objetivo de garantizar que sólo pueda haber un cliente sentado en él.

Respecto a los **eventos de sincronización**, en este problema existe una interacción directa entre el cliente que pasa a la sala del barbero, es decir, el proceso de *despertar* al barbero, y la notificación del barbero al cliente cuando ya ha terminado su trabajo. El primer evento se ha modelado mediante un semáforo binario denominado *barbero*, ya que está asociado al proceso de despertar al barbero por parte de un cliente. El segundo evento se ha modelado, igualmente, con otro semáforo binario denominado *fin*, ya que está asociado al proceso de notificar el final del trabajo a un cliente por parte del barbero. Ambos semáforos se inicializan a 0.

En la figura 2.19 se muestra el pseudocódigo de una posible solución al problema del barbero dormilón. Como se puede apreciar, la parte relativa al **proceso barbero** es trivial, ya que éste permanece de manera pasiva a la espera de un nuevo cliente, es decir, permanece bloqueado mediante *wait* sobre el semáforo *barbero*. Cuando terminado de cortar, el barbero notificará dicha finalización al cliente mediante *signal* sobre el semáforo *fin*. Evidentemente, el cliente estará bloqueado por *wait*.

El pseudocódigo del **proceso cliente** es algo más complejo, ya que hay que controlar si un cliente puede o no pasar a la sala de espera. Para ello, en primer lugar se ha de comprobar si todas las sillas están ocupadas, es decir, si el valor de la variable compartida *num_clientes* a llegado al máximo, es decir, a las N sillas que conforman la sala de espera. En tal caso, el cliente habrá de abandonar la barbería. Note cómo el acceso a *num_clientes* se gestiona mediante *mutex*.

Si hay alguna silla disponible, entonces el cliente esperará su turno. En otras palabras, esperará el acceso al sillón del barbero. Esta situación se modela ejecutando *wait* sobre el semáforo *sillón*. Si un cliente se sienta en el sillón, entonces deja una silla disponible.

A continuación, el cliente sentado en el sillón despertará al barbero y esperará a que éste termine su trabajo. Finalmente, el cliente abandonará la barbería liberando el sillón para el siguiente cliente (si lo hubiera).



Figura 2.20: Patrón *rendezvous* para la sincronización de dos procesos en un determinado punto.

2.3.6. Los caníbales comensales

En una tribu de caníbales todos comen de la misma olla, la cual puede albergar N raciones de *comida*. Cuando un caníbal quiere comer, simplemente se sirve de la olla común, a no ser que esté vacía. En ese caso, el caníbal despierta al cocinero de la tribu y espera hasta que éste haya rellenado la olla.

En este problema, los **eventos de sincronización** son dos:

- Si un caníbal que quiere comer se encuentra con la olla vacía, entonces se lo notifica al cocinero para que éste cocine.
- Cuando el cocinero termina de cocinar, entonces se lo notifica al caníbal que lo despertó previamente.

Un posible planteamiento para solucionar este problema podría girar en torno al uso de un semáforo que controlara el número de raciones disponibles en un determinado momento (de manera similar al problema del buffer limitado). Sin embargo, este planteamiento no facilita la notificación al cocinero cuando la olla esté vacía, ya que no es deseable acceder al valor interno de un semáforo y, en función del mismo, actuar de una forma u otra.

Una alternativa válida consiste en utilizar el patrón **marcador** para controlar el número de raciones de la olla mediante una variable compartida. Si ésta alcanza el valor de 0, entonces el caníbal podría despertar al cocinero.

La sincronización entre el caníbal y el cocinero se realiza mediante **rendezvous**:

1. El caníbal intenta obtener una ración.
2. Si no hay raciones en la olla, el caníbal despierta al cocinero.
3. El cocinero cocina y rellena la olla.
4. El cocinero notifica al caníbal.
5. El caníbal come.

La solución planteada hace uso de los siguientes elementos:

- **Num_Raciones**, variable compartida que contiene el número de raciones disponibles en la olla en un determinado instante de tiempo.
- **Mutex**, semáforo binario que controla el acceso a *Num_Raciones*.
- **Empty**, que controla cuando la olla se ha quedado vacía.
- **Full**, que controla cuando la olla está llena.

En el siguiente listado se muestra el pseudocódigo del proceso **cocinero**, el cual es muy simple ya que se basa en esperar la llamada del caníbal, cocinar y notificar de nuevo al caníbal.

Listado 2.15: Proceso Cocinero.

```
1 while (1) {  
2     wait_sem(empty);  
3     cocinar();  
4     signal_sem(full);  
5 }
```

El pseudocódigo del proceso **caníbal** es algo más complejo, debido a que ha de consultar el número de raciones disponibles en la olla antes de comer.

Listado 2.16: Proceso Caníbal.

```
1 while (1) {  
2     wait(mutex);  
3  
4     if (num_raciones == 0) {  
5         signal(empty);  
6         wait(full);  
7         num_raciones = N;  
8     }  
9  
10    num_raciones--;  
11  
12    signal(mutex);  
13  
14    comer();  
15 }
```

Como se puede apreciar, el proceso caníbal comprueba el número de raciones disponibles en la olla (línea 4). Si no hay, entonces despierta al cocinero (línea 5) y espera a que éste rellene la olla (línea 6). Estos dos eventos de sincronización guían la evolución de los procesos involucrados.

Note cómo el proceso *caníbal* modifica el número de raciones asignando un valor constante. El lector podría haber optado porque fuera el cocinero el que modificara la variable, pero desde un punto de vista práctico es más eficiente que lo haga el caníbal, ya que tiene adquirido el acceso a la variable mediante el semáforo *mutex* (líneas 2 y 12).

Posteriormente, el caníbal decrementa el número de raciones (línea [12](#)) e invierte un tiempo en comer (línea [14](#)).

2.3.7. El problema de Santa Claus

Santa Claus pasa su tiempo de descanso, durmiendo, en su casa del Polo Norte. Para poder despertarlo, se ha de cumplir una de las dos condiciones siguientes:

1. Que todos los renos de los que dispone, nueve en total, hayan vuelto de vacaciones.
2. Que algunos de sus duendes necesiten su ayuda para fabricar un juguete.

Para permitir que Santa Claus pueda descansar, los duendes han acordado despertarlo si tres de ellos tienen problemas a la hora de fabricar un juguete. En el caso de que un grupo de tres duendes estén siendo ayudados por Santa, el resto de los duendes con problemas tendrán que esperar a que Santa termine de ayudar al primer grupo.

En caso de que haya duendes esperando y todos los renos hayan vuelto de vacaciones, entonces Santa Claus decidirá preparar el trineo y repartir los regalos, ya que su entrega es más importante que la fabricación de otros juguetes que podría esperar al año siguiente. El último reno en llegar ha de despertar a Santa mientras el resto de renos esperan antes de ser enganchados al trineo.

Para **solucionar** este problema, se pueden distinguir tres procesos básicos: i) Santa Claus, ii) duende y iii) reno. Respecto a los recursos compartidos, es necesario controlar el número de duendes que, en un determinado momento, necesitan la ayuda de Santa y el número de renos que, en un determinado momento, están disponibles. Evidentemente, el acceso concurrente a estas variables ha de estar controlado por un semáforo binario.

Respecto a los **eventos de sincronización**, será necesario disponer de mecanismos para despertar a Santa Claus, notificar a los renos que se han de enganchar al trineo y controlar la espera por parte de los duendes cuando otro grupo de duendes esté siendo ayudado por Santa Claus.

En resumen, se utilizarán las siguientes estructuras para plantear la solución del problema:

- **Duendes**, variable compartida que contiene el número de duendes que necesitan la ayuda de Santa en un determinado instante de tiempo.

- **Renos**, variable compartida que contiene el número de renos que han vuelto de vacaciones y están disponibles para viajar.
- **Mutex**, semáforo binario que controla el acceso a *Duendes* y *Renos*.
- **SantaSem**, semáforo binario utilizado para despertar a Santa Claus.
- **RenosSem**, semáforo contador utilizado para notificar a los renos que van a emprender el viaje en trineo.
- **DuendesSem**, semáforo contador utilizado para notificar a los duendes que Santa los va a ayudar.
- **DuendesMutex**, semáforo binario para controlar la espera de duendes cuando Santa está ayudando a otros.

En el siguiente listado se muestra el pseudocódigo del proceso **Santa Claus**. Como se puede apreciar, Santa está durmiendo a la espera de que lo despierten (línea [3]). Si lo despiertan, será porque los duendes necesitan su ayuda o porque todos los renos han vuelto de vacaciones. Por lo tanto, Santa tendrá que comprobar cuál de las dos condiciones se ha cumplido.

Listado 2.17: Proceso Santa Claus.

```
1 while (1) {
2     /* Espera a ser despertado */
3     wait(santa_sem);
4
5     wait(mutex);
6
7     /* Todos los renos preparados? */
8     if (renos == TOTAL_RENOS) {
9         prepararTrineo();
10        /* Notificar a los renos... */
11        for (i = 0; i < TOTAL_RENOS; i++)
12            signal(renos_sem);
13    }
14    else
15        if (duendes == NUM_DUENDES_GRUPO) {
16            ayudarDuendes();
17            /* Notificar a los duendes... */
18            for (i = 0; i < NUM_DUENDES_GRUPO; i++)
19                signal(duendes_sem);
20        }
21
22    signal(mutex);
23 }
```

Si todos los renos están disponibles (línea [8]), entonces Santa preparará el trineo y notificará a todos los renos (líneas [9-12]). Note cómo tendrá que hacer tantas notificaciones (*signal*) como renos haya disponibles. Si hay suficientes duendes para que sean ayudados (líneas [15-20]), entonces Santa los ayudará, notificando esa ayuda de manera explícita (*signal*) mediante el semáforo *DuendesSem*.

El proceso **reno** es bastante sencillo, ya que simplemente despierta a Santa cuando todos los renos estén disponibles y, a continuación, esperar la notificación de Santa. Una vez más, el acceso a la variable compartida *renos* se controla mediante el semáforo binario *mutex*.

Listado 2.18: Proceso reno.

```
1 vacaciones();
2
3 wait(mutex);
4
5 renos += 1;
6 /* Todos los renos listos? */
7 if (renos == TOTAL_RENOS)
8     signal(santa_sem);
9
10 signal(mutex);
11
12 /* Esperar la notificación de Santa */
13 wait(renos_sem);
14 engancharTrineo();
```

Finalmente, en el proceso **duende** se ha de controlar la formación de grupos de duendes de tres componentes antes de despertar a Santa (línea [6]). Si se ha alcanzado el número mínimo para poder despertar a Santa, entonces se le despierta mediante *signal* sobre el semáforo *SantaSem* (línea [7]). Si no es así, es decir, si otro duende necesita ayuda pero no se ha llegado al número mínimo de duendes para despertar a Santa, entonces el semáforo *DuendesMutex* se libera (línea [9]).

El duende invocará a *obtenerAyuda* y esperará a que Santa notifique dicha ayuda mediante *DuendesSem*. Note cómo después de solicitar ayuda, el duende queda a la espera de la notificación de Santa.

Listado 2.19: Proceso duende.

```
1 wait(duendes_mutex);
2 wait(mutex);
3
4 duendes += 1;
5 /* Está completo el grupo de duendes? */
6 if (duendes == NUM_DUENDES_GRUPO)
7     signal(santa_sem);
8 else
9     signal(duendes_mutex);
10
11 signal(mutex);
12
13 wait(duendes_sem);
14 obteniendoAyuda();
15
16 wait(mutex);
17
18 duendes -= 1;
19 /* Nuevo grupo de duendes? */
20 if (duendes == 0)
21     signal(duendes_mutex);
22
23 signal(mutex);
```



3

Capítulo

Paso de Mensajes

En este capítulo se discute el concepto de **paso de mensajes** como mecanismo básico de sincronización entre procesos. Al contrario de lo que ocurre con los semáforos, el paso de mensajes ya maneja, de manera implícita, el intercambio de información entre procesos, sin tener que recurrir a otros elementos como por ejemplo el uso de segmentos de memoria compartida.

El paso de mensajes está orientado a los sistemas distribuidos y la sincronización se alcanza mediante el uso de dos primitivas básicas: i) el envío y ii) la recepción de mensajes. Así, es posible establecer esquemas basados en la recepción bloqueante de mensajes, de manera que un proceso se quede bloqueado hasta que reciba un mensaje determinado.

Además de discutir los conceptos fundamentales de este mecanismo de sincronización, en este capítulo se estudian las primitivas POSIX utilizadas para establecer un sistema de paso de mensajes mediante las denominadas **colas de mensajes POSIX** (*POSIX Message Queues*).

Finalmente, este capítulo plantea algunos problemas clásicos de sincronización en los que se utiliza el paso de mensajes para obtener una solución que gestione la ejecución concurrente de los mismos y el problema de la sección crítica, estudiado en el capítulo 1.

3.1. Conceptos básicos

Dentro del ámbito de la programación concurrente, el paso de mensajes representa un mecanismo básico de comunicación entre procesos basado, principalmente, en el envío y recepción de mensajes.

En realidad, el paso de mensajes representa una abstracción del concepto de comunicación que manejan las personas, representado por mecanismos concretos como por ejemplo el correo electrónico. La **comunicación entre procesos** (*InterProcess Communication*, IPC) se basa en esta filosofía. El proceso emisor genera un mensaje como un bloque de información que responde a un determinado formato. El sistema operativo copia el mensaje del espacio de direcciones del emisor en el espacio de direcciones del proceso receptor, tal y como se muestra en la figura 3.1.

En el contexto del paso de mensajes, es importante recordar que un emisor no puede copiar información, directamente, en el espacio de direcciones de otro proceso receptor. En realidad, esta posibilidad está reservada para el software en modo *supervisor* del sistema operativo.

En su lugar, el proceso emisor pide que el sistema operativo entregue un mensaje en el espacio de direcciones del proceso receptor utilizando, en algunas situaciones, el identificador único o PID del proceso receptor. Así, el sistema operativo hace uso de ciertas operaciones de copia para alcanzar este objetivo: i) recupera el mensaje del espacio de direcciones del emisor, ii) lo coloca en un buffer del sistema operativo y iii) copia el mensaje de dicho buffer en el espacio de direcciones del receptor.

A diferencia de lo que ocurre con los sistemas basados en el uso de semáforos y segmentos de memoria compartida, en el caso de los sistemas en paso de mensajes el sistema operativo facilita los mecanismos de comunicación y sincronización sin necesidad de utilizar objetos de memoria compartida. En otras palabras, el paso de mensajes permite que los procesos se envíen información y, al mismo tiempo, se sincronicen.

Desde un punto de vista general, los sistemas de paso de mensajes se basan en el uso de dos **primitivas de comunicación**:

- **Envío** (*send*), que permite el envío de información por parte de un proceso.
- **Recepción** (*receive*), que permite la recepción de información por parte de un proceso.

Típicamente, y como se discutirá en la sección 3.1.3, la **sincronización** se suele plantear en términos de llamadas bloqueantes, cuya principal implicación es el bloqueo de un proceso hasta que se satisfaga una cierta restricción (por ejemplo, la recepción de un mensaje).

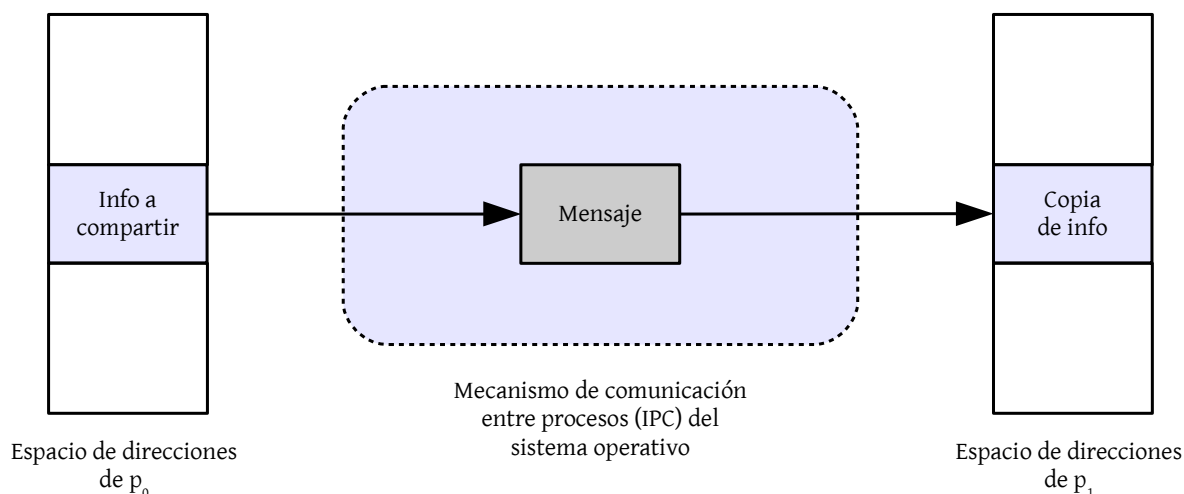


Figura 3.1: Uso de mensajes para compartir información entre procesos.

3.1.1. El concepto de buzón o cola de mensajes

Teniendo en cuenta la figura 3.1, el lector podría pensar que el envío de un mensaje podría modificar el espacio de direcciones de un proceso receptor de manera espontánea sin que el propio receptor estuviera al corriente. Esta situación se puede evitar si no se realiza una copia en el espacio de direcciones del proceso receptor hasta que éste no la solicite de manera explícita (mediante una primitiva de recepción).

Ante esta problemática, el sistema operativo puede almacenar mensajes de entrada en una cola o buzón de mensajes que actúa como buffer previo a la copia en el espacio de direcciones del receptor. La figura 3.2 muestra de manera gráfica la comunicación existente entre dos procesos mediante el uso de un buzón o **cola de mensajes**.

3.1.2. Aspectos de diseño en sistemas de mensajes

En esta sección se discuten algunos aspectos de diseño [13] que marcan el funcionamiento básico de un sistema basado en mensajes. Es muy importante conocer qué mecanismo de sincronización se utilizará a la hora de manejar las primitivas *send* y *receive*, con el objetivo de determinar si las llamadas a dichas primitivas serán bloqueantes.

Sincronización

La comunicación de un mensaje entre dos procesos está asociada a un nivel de sincronización ya que, evidentemente, un proceso *A* no puede recibir un mensaje hasta que otro proceso *B* lo envía. En este contexto, se plantea la necesidad de asociar un comportamiento a un

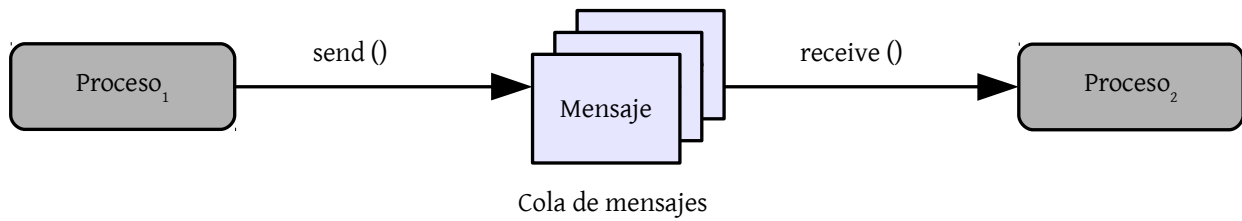


Figura 3.2: Esquema de paso de mensajes basado en el uso de colas o buzones de mensajes.

proceso cuando envía y recibe un mensaje.

Por ejemplo, cuando un proceso recibe un mensaje mediante una primitiva de **recepción**, existen dos posibilidades:

- Si el mensaje ya fue enviado por un proceso emisor, entonces el receptor puede obtener el mensaje y continuar su ejecución.
- Si no existe un mensaje por obtener, entonces se plantean a su vez dos opciones:
 - Que el proceso receptor se bloquee hasta que reciba el mensaje.
 - Que el proceso receptor continúe su ejecución obviando, al menos temporalmente, el intento de recepción.

Del mismo modo, la operación de **envío** de mensajes se puede plantear de dos formas posibles:

- Que el proceso emisor se bloquee hasta que el receptor reciba el mensaje.
- Que el proceso emisor no se bloquee y se aisle, al menos temporalmente¹, del proceso de recepción del mensaje

En el ámbito de la programación concurrente, la **primitiva send** se suele utilizar con una naturaleza no bloqueante, con el objetivo de que un proceso pueda continuar su ejecución y, en consecuencia, pueda progresar en su trabajo. Sin embargo, este esquema puede fomentar prácticas no deseables, como por ejemplo que un proceso envíe mensajes de manera continuada sin que se produzca ningún tipo de penalización, asociada a su vez al bloqueo del proceso emisor. Desde el punto de vista del programador, este esquema hace que el propio programador sea el responsable de comprobar si un mensaje se recibió de manera adecuada, complicando así la lógica del programa.

En el caso de la **primitiva receive**, la versión bloqueante suele ser la elegida con el objetivo de sincronizar procesos. En otras palabras,

¹Considere el uso de un esquema basado en retrollamadas o notificaciones

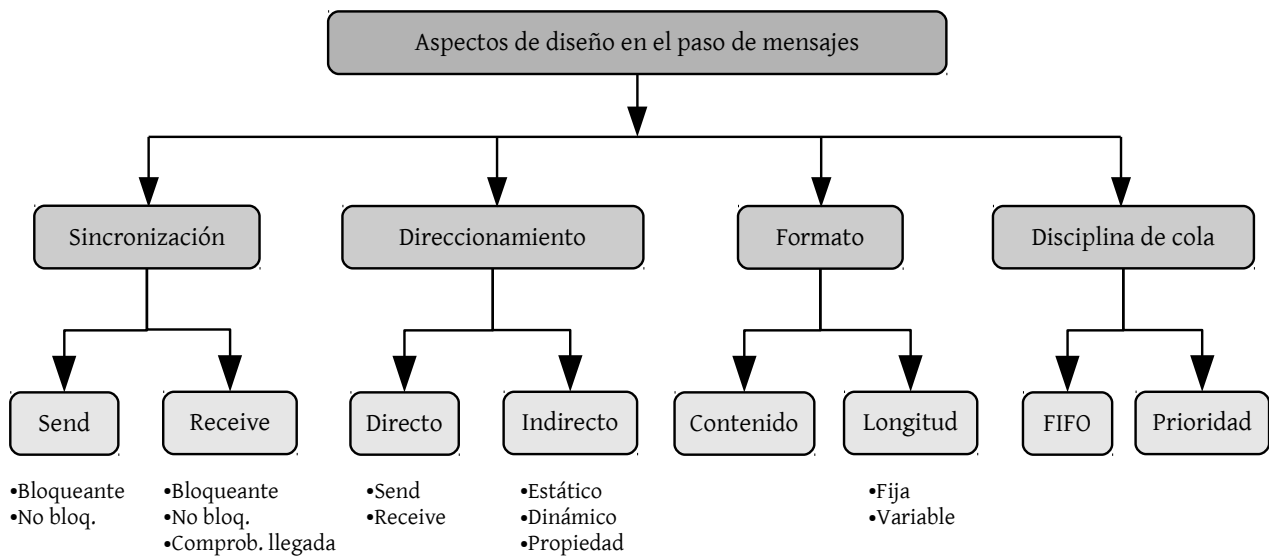


Figura 3.3: Aspectos básicos de diseño en sistemas de paso de mensajes para la comunicación y sincronización entre procesos.

un proceso que espera un mensaje suele necesitar la información contenida en el mismo para continuar con su ejecución. Un problema que surge con este esquema está derivado de la pérdida de un mensaje, ya que el proceso receptor se quedaría bloqueado. Dicho problema se puede abordar mediante una naturaleza no bloqueante.

Direccionamiento

Recuperación por tipo

También es posible plantear un esquema basado en la recuperación de mensajes por tipo, es decir, una recuperación selectiva atendiendo al contenido del propio mensaje.

Desde el punto de vista del envío y recepción de mensajes, resulta muy natural enviar y recibir mensajes de manera selectiva, es decir, especificando de manera explícita qué proceso enviará un mensaje y qué proceso lo va a recibir. Dicha información se podría especificar en las propias primitivas de envío y recepción de mensajes.

Este tipo de direccionamiento de mensajes recibe el nombre de **direccionamiento directo**, es decir, la primitiva *send* contiene un identificador de proceso vinculado al proceso emisor. En el caso de la primitiva *receive* se plantean dos posibilidades:

1. Designar de manera **explícita** al proceso emisor para que el proceso receptor sepa de quién recibe mensajes. Esta alternativa suele ser la más eficaz para procesos concurrentes cooperativos.
2. Hacer uso de un direccionamiento directo **implícito**, con el objetivo de modelar situaciones en las que no es posible especificar, de manera previa, el proceso de origen. Un posible ejemplo sería el proceso responsable del servidor de impresión.

El otro esquema general es el **direccionamiento indirecto**, es decir, mediante este esquema los mensajes no se envían directamente por un emisor a un receptor, sino que son enviados a una estructura de datos compartida, denominada buzón o cola de mensajes, que almacenan los mensajes de manera temporal. De este modo, el modelo de comunicación se basa en que un proceso envía un mensaje a la cola y otro proceso toma dicho mensaje de la cola.

La principal ventaja del direccionamiento indirecto respecto al directo es la flexibilidad, ya que existe un desacoplamiento entre los emisores y receptores de mensajes, posibilitando así diversos esquemas de comunicación y sincronización:

- uno a uno, para llevar a cabo una comunicación privada entre procesos.
- muchos a uno, para modelar situaciones cliente/servidor. En este caso, el buzón se suele conocer como *puerto*.
- uno a muchos, para modelar sistemas de *broadcast* o difusión de mensajes.
- muchos a muchos, para que múltiples clientes puedan acceder a un servicio concurrente proporcionado por múltiples servidores.

Finalmente, es importante considerar que la asociación de un proceso con respecto a un buzón puede ser estática o dinámica. Así mismo, el concepto de **propiedad** puede ser relevante para establecer relaciones entre el ciclo de vida de un proceso y el ciclo de vida de un buzón asociado al mismo.

Formato de mensaje

El formato de un mensaje está directamente asociado a la funcionalidad proporcionada por la biblioteca de gestión de mensajes y al ámbito de aplicación, entendiendo *ámbito* como su uso en una única máquina o en un sistema distribuido. En algunos sistemas de paso de mensajes, los mensajes tienen una **longitud fija** con el objetivo de no sobrecargar el procesamiento y almacenamiento de información. En otros, los mensajes pueden tener una **longitud variable** con el objetivo de incrementar la flexibilidad.

En la figura 3.4 se muestra el posible formato de un mensaje típico, distinguiendo entre la cabecera y el cuerpo del mensaje. Como se puede apreciar, en dicho mensaje van incluidos los identificadores únicos de los procesos emisor y receptor, conformando así un direccionamiento directo explícito.

Disciplina de cola

La disciplina de cola se refiere a la implementación subyacente que rige el comportamiento interno de la cola. Típicamente, las colas o buzones de mensajes se implementan haciendo uso de una cola **FIFO**

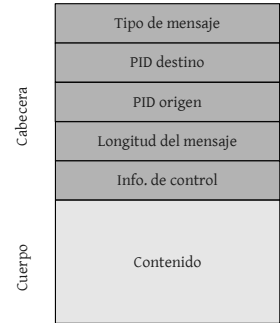


Figura 3.4: Formato típico de un mensaje, estructurado en cabecera y cuerpo. Como se puede apreciar en la imagen, la cabecera contiene información básica para, por ejemplo, establecer un esquema dinámico asociado a la longitud del contenido del mensaje.

(*First-In First-Out*), es decir, los mensajes primeros en salir son aquellos que llegaron en primer lugar.

Sin embargo, es posible utilizar una disciplina basada en el uso de un esquema basado en **prioridades**, asociando un valor numérico de importancia a los propios mensajes para establecer políticas de ordenación en base a los mismos.

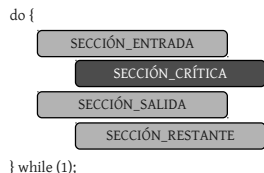


Figura 3.5: Visión gráfica del problema general de la sección crítica, distinguiendo la sección de entrada, la propia sección crítica, la sección de salida y la sección restante.

3.1.3. El problema de la sección crítica

El problema de la sección crítica, introducido en la sección 1.2.2, se puede resolver fácilmente utilizando el mecanismo del paso de mensajes, tal y como muestra la figura 3.6.



Figura 3.6: Mecanismo de exclusión mutua basado en el uso de paso de mensajes.

Básicamente, es posible modelar una solución al problema de la sección crítica mediante la primitiva *receive* bloqueante. Así, los procesos comparten un único buzón, inicializado con un único mensaje, sobre el que envían y reciben mensajes. Un proceso que desee acceder a la sección crítica accederá al buzón mediante la primitiva *receive* para obtener un mensaje. En esta situación se pueden distinguir dos posibles casos:

1. Si el buzón está vacío, entonces el proceso se bloquea. Esta situación implica que, de manera previa, otro proceso accedió al buzón y recuperó el único mensaje del mismo. El primer proceso se quedará bloqueado hasta que pueda recuperar un mensaje del buzón.
2. Si el buzón tiene un mensaje, entonces el proceso lo recupera y puede acceder a la sección crítica.

En cualquier caso, el proceso ha de devolver el mensaje al buzón en su sección de salida para garantizar que el resto de procesos puedan

acceder a su sección crítica. En esencia, el único mensaje del buzón se comporta como un **token** o testigo que gestiona el acceso exclusivo a la sección crítica. Es importante destacar que si existen varios procesos bloqueados a la espera de un mensaje en el buzón, entonces sólo uno de ellos se desbloqueará cuando otro proceso envíe un mensaje al buzón. No es posible conocer a priori qué proceso se desbloqueará aunque, típicamente, las colas de mensajes se implementan mediante colas FIFO (*first-in first-out*), por lo que aquellos procesos que se bloqueen en primer lugar serán los primeros en desbloquearse.

Desde un punto de vista semántico, el uso de un mecanismo basado en el paso de mensajes para modelar el problema de la sección crítica es idéntico al uso de un mecanismo basado en **semáforos**. Si se utiliza un semáforo para gestionar el acceso a la sección crítica, éste ha de inicializarse con un valor igual al número de procesos que pueden acceder a la sección crítica de manera simultánea. Típicamente, este valor será igual a uno para proporcionar un acceso exclusivo.

Si se utiliza el paso de mensajes, entonces el programador tendrá que *inicializar* el buzón con tantos mensajes como procesos puedan acceder a la sección crítica de manera simultánea (uno para un acceso exclusivo). La diferencia principal reside en que la inicialización es una operación propia del semáforo, mientras que en el paso de mensajes se ha de utilizar la primitiva *send* para rellenar el buzón y, de este modo, llevar a cabo la *inicialización* del mismo.

3.2. Implementación

En esta sección se discuten las **primitivas POSIX** más importantes relativas al paso de mensajes. Para ello, POSIX proporciona las denominadas colas de mensajes (*POSIX Message Queues*).

En el siguiente listado de código se muestra la primitiva *mq_open()*, utilizada para **abrir** una cola de mensajes existente o crear una nueva. Como se puede apreciar, dicha primitiva tiene dos versiones. La más completa incluye los permisos y una estructura para definir los atributos, además de establecer el nombre de la cola de mensajes y los *flags*.

Si la primitiva *mq_open()* abre o crea una cola de mensajes de manera satisfactoria, entonces devuelve el descriptor de cola asociada a la misma. Si no es así, devuelve `-1` y establece *errno* con el error generado.

Compilación

Utilice los *flags* adecuados en cada caso con el objetivo de detectar la mayor cantidad de errores en tiempo de compilación (e.g. escribir sobre una cola de mensajes de sólo lectura).



Las colas de mensajes en POSIX han de empezar por el carácter '/', como por ejemplo '/BuzonMensajes1'.

Al igual que ocurre con los semáforos nombrados, las colas de mensajes en POSIX se basan en la definición de un nombre específico para llevar a cabo la operación de apertura o creación de una cola.

Listado 3.1: Primitiva `mq_open` en POSIX.

```

1 #include <fcntl.h>      /* Para constantes O_* */
2 #include <sys/stat.h>   /* Para los permisos */
3 #include <mqueue.h>
4
5 /* Devuelve el descriptor de la cola de mensajes */
6 /* o -1 si hubo error (ver errno) */
7 mqd_t mq_open (
8     const char *name,      /* Nombre de la cola */
9     int oflag              /* Flags (menos O_CREAT) */
10 );
11 mqd_t mq_open (
12     const char *name,      /* Nombre de la cola */
13     int oflag              /* Flags (incluyendo O_CREAT) */
14     mode_t perms,         /* Permisos */
15     struct mq_attr *attr  /* Atributos (o NULL) */
16 );

```

En el siguiente listado de código se muestra un ejemplo específico de creación de una cola de mensajes en POSIX. En las líneas 6-7 se definen los atributos básicos del buzón:

- `mq_maxmsg`, que define el máximo número de mensajes.
- `mq_msgsize`, que establece el tamaño máximo de mensaje.

Además de la posibilidad de establecer aspectos básicos relativos a una cola de mensajes a la hora de crearla, como el tamaño máximo del mensaje, es posible obtenerlos e incluso establecerlos de manera explícita, como se muestra en el siguiente listado de código.

Listado 3.2: Obtención/modificación de atributos en una cola de mensajes en POSIX.

```

1 #include <mqueue.h>
2
3 int mq_getattr (
4     mqd_t mqdes,          /* Descriptor de la cola */
5     struct mq_attr *attr  /* Atributos */
6 );
7
8 int mq_setattr (
9     mqd_t mqdes,          /* Descriptor de la cola */
10    struct mq_attr *newattr, /* Nuevos atributos */
11    struct mq_attr *oldattr  /* Antiguos atributos */
12 );

```

Por otra parte, en la línea `(10)` se lleva a cabo la operación de creación de la cola de mensajes, identificada por el nombre `/prueba` y sobre la que el proceso que la crea podrá llevar a cabo operaciones de lectura (recepción) y escritura (envío). Note cómo la estructura relativa a los atributos previamente comentados se pasa por referencia. Finalmente, en las líneas `(14-17)` se lleva a cabo el control de errores básicos a la hora de abrir la cola de mensajes.

Las primitivas relativas al **cierre** y a la **eliminación** de una cola de mensajes son muy simples y se muestran en el siguiente listado de código. La primera de ellas necesita el descriptor de la cola, mientras que la segunda tiene como parámetro único el nombre asociado a la misma.

Listado 3.3: Ejemplo de creación de una cola de mensajes en POSIX.

```
1 /* Descriptor de cola de mensajes */
2 mqd_t qHandler;
3 int rc;
4 struct mq_attr mqAttr;      /* Estructura de atributos */
5
6 mqAttr.mq_maxmsg = 10;      /* Máximo no de mensajes */
7 mqAttr.mq_msgsize = 1024;   /* Tamaño máximo de mensaje */
8
9 /* Creación del buzón */
10 qHandler = mq_open('/prueba', O_RDWR | O_CREAT,
11                    S_IWUSR | S_IRUSR, &mqAttr);
12
13 /* Manejo de errores */
14 if (qHandler == -1) {
15     fprintf(stderr, "%s\n", strerror(errno));
16     exit(EXIT_FAILURE);
17 }
```

Desde el punto de vista de la programación concurrente, las primitivas más relevantes son las de **envío y recepción** de mensajes, ya que permiten no sólo sincronizar los procesos sino también compartir información entre los mismos. En el siguiente listado de código se muestra la signatura de las primitivas POSIX *send* y *receive*.

Listado 3.4: Primitivas de cierre y eliminación de una cola de mensajes en POSIX.

```
1 #include <mqueue.h>
2
3 int mq_close (
4     mqd_t mqdes      /* Descriptor de la cola de mensajes */
5 );
6
7 int mq_unlink (
8     const char *name  /* Nombre de la cola */
9 );
```

Por una parte, la primitiva *send* permite el envío del mensaje definido por *msg_ptr*, y cuyo tamaño se especifica en *msg_len*, a la cola asociada al descriptor especificado por *mqdes*. Es posible asociar una prioridad a los mensajes que se envían a una cola mediante el parámetro *msg_prio*.

Por otra parte, la primitiva *receive* permite recibir mensajes de una cola. Note cómo los parámetros son prácticamente los mismos que los de la primitiva *send*. Sin embargo, ahora *msg_ptr* representa el buffer que almacenará el mensaje a recibir y *msg_len* es el tamaño de dicho buffer. Además, la primitiva de recepción implica obtener la prioridad asociada al mensaje a recibir, en lugar de especificarla a la hora de enviarlo.

Listado 3.5: Primitivas de envío y recepción en una cola de mensajes en POSIX.

```

1 #include <mqueue.h>
2
3 int mq_send (
4     mqd_t mqdes,          /* Descriptor de la cola */
5     const char *msg_ptr,  /* Mensaje */
6     size_t msg_len,       /* Tamaño del mensaje */
7     unsigned msg_prio     /* Prioridad */
8 );
9
10 ssize_t mq_receive (
11     mqd_t mqdes,          /* Descriptor de la cola */
12     char *msg_ptr,        /* Buffer para el mensaje */
13     size_t msg_len,       /* Tamaño del buffer */
14     unsigned *msg_prio    /* Prioridad o NULL */
15 );

```

El siguiente listado de código muestra un ejemplo muy representativo del uso del paso de mensajes en POSIX, ya que refleja la **recepción bloqueante**, una de las operaciones más representativas de este mecanismo.

En las líneas [6-7] se refleja el uso de la primitiva *receive*, utilizando *buffer* como variable receptora del contenido del mensaje a recibir y *prioridad* para obtener el valor de la misma. Recuerde que la semántica de las operaciones de una cola, como por ejemplo que la recepción sea no bloqueante, se definen a la hora de su creación, tal y como se mostró en uno de los listados anteriores, mediante la estructura *mq_attr*. Sin embargo, es posible modificar y obtener los valores de configuración asociados a una cola mediante las primitivas *mq_setattr* y *mq_getattr*, respectivamente.

Listado 3.6: Ejemplo de recepción bloqueante en una cola de mensajes en POSIX.

```

1 mqd_t qHandler;
2 int rc;
3 unsigned int prioridad;
4 char buffer[2048];
5
6 rc = mq_receive(qHandler, buffer,
7     sizeof(buffer), &prioridad);
8
9 if (rc == -1)
10     fprintf(stderr, "%s\n", strerror(errno));
11 else
12     printf ("Recibiendo mensaje: %s\n", buffer);

```



El buffer de recepción de mensajes ha de tener un tamaño mayor o igual a `mq_msgsize`, mientras que el buffer de envío de un mensaje ha de tener un tamaño menor o igual a `mq_msgsize`.

En el anterior listado también se expone cómo obtener información relativa a posibles errores mediante ***errno***. En este contexto, el fichero de cabecera *errno.h*, incluido en la biblioteca estándar del lenguaje C, define una serie de macros para informar sobre condiciones de error, a través de una serie de códigos de error, almacenados en un espacio estático denominado *errno*.

De este modo, algunas primitivas modifican el valor de este espacio cuando detectan determinados errores. En el ejemplo de recepción de mensajes, si *receive* devuelve un código de error -1, entonces el programador puede obtener más información relativa al error que se produjo, como se aprecia en las líneas [9-10].

El **envío de mensajes**, en el ámbito de la programación concurrente, suele mantener una semántica no bloqueante, es decir, cuando un proceso envía un mensaje su flujo de ejecución continúa inmediatamente después de enviar el mensaje. Sin embargo, si la cola POSIX ya estaba llena de mensajes, entonces hay que distinguir entre dos casos:

1. Por defecto, un proceso se bloqueará al enviar un mensaje a una cola llena, hasta que haya espacio disponible para encolarlo.
2. Si el flag *O_NONBLOCK* está activo en la descripción de la cola de mensajes, entonces la llamada a *send* fallará y devolverá un código de error.

El siguiente listado de código muestra un **ejemplo de envío** de mensajes a través de la primitiva *send*. Como se ha comentado anteriormente, esta primitiva de envío de mensajes se puede comportar de una manera no bloqueante cuando la cola está llena de mensajes (ha alcanzado el número máximo definido por *mq_maxmsg*).

Listado 3.7: Ejemplo de envío en una cola de mensajes en POSIX.

```
1 mqd_t qHandler;
2 int rc;
3 unsigned int prioridad;
4 char buffer[512];
5
6 sprintf (buffer, "[ Saludos de %d ]", getpid());
7 mq_send(qHandler, buffer, sizeof(buffer), 1);
```

Llegados a este punto, el lector se podría preguntar cómo es posible incluir tipos o **estructuras de datos específicas** de un determinado dominio con el objetivo de intercambiar información entre varios procesos mediante el uso del paso de mensajes. Esta duda se podría plantear debido a que tanto la primitiva de envío como la primitiva

En POSIX...

En el estándar POSIX, las primitivas *mq_timedsend* y *mq_timedreceive* permiten asociar *timeouts* para especificar el tiempo máximo de bloqueo al utilizar las primitivas *send* y *receive*, respectivamente.

de recepción de mensajes manejan cadenas de caracteres (punteros a *char*) para gestionar el contenido del mensaje.

En realidad, este enfoque, extendido en diversos ámbitos del estándar POSIX, permite manejar cualquier tipo de datos de una manera muy sencilla, ya que sólo es necesario aplicar los moldes necesarios para enviar cualquier tipo de información. En otras palabras, el tipo de datos *char ** se suele utilizar como buffer genérico, posibilitando el uso de cualquier tipo de datos definido por el usuario.



Muchos APIs utilizan *char ** como buffer genérico. Simplemente, considere un puntero a buffer, junto con su tamaño, a la hora de manejar estructuras de datos como contenido de mensaje en las primitivas *send* y *receive*.

El siguiente listado de código muestra un ejemplo representativo de esta problemática. Como se puede apreciar, en dicho ejemplo el buffer utilizado es una estructura del tipo *TData*, definida por el usuario. Note cómo al usar las primitivas *send* y *receive* se hace uso de los moldes correspondientes para cumplir con la especificación de dichas primitivas.

Listado 3.8: Uso de tipos de datos específicos para manejar mensajes.

```
1 typedef struct {
2     int valor;
3 } TData;
4
5 TData buffer;
6
7 /* Atributos del buzón */
8 mqAttr.mq_maxmsg = 10;
9 mqAttr.mq_msgsize = sizeof(TData);
10
11 /* Recepción */
12 mq_receive(qHandler, (char *)&buffer,
13             sizeof(buffer), &prioridad);
14 /* Envío */
15 mq_send(qHandler, (const char *)&buffer,
16          sizeof(buffer), 1);
```

Finalmente, el estándar POSIX también define la primitiva *mq_notify* en el contexto de las colas de mensajes con el objetivo de **notificar a un proceso** o hilo, mediante una señal, cuando llega un mensaje.

Básicamente, esta primitiva permite que cuando un proceso o un hilo se registre, éste reciba una señal cuando llegue un mensaje a una cola vacía especificada por *mqdes*, a menos que uno o más procesos o hilos ya estén bloqueados mediante una llamada a *mq_receive*. En esta situación, una de estas llamadas a dicha primitiva retornará en su lugar.

Listado 3.9: Primitiva mq_notify en POSIX.

```
1 #include <mqueue.h>
2
3 int mq_notify (
4     /* Descriptor de la cola */
5     mqd_t mqdes,
6     /* Estructura de notificación
7     Contiene funciones de retrollamada
8     para notificar la llegada de un nuevo mensaje */
9     const struct sigevent *sevp
10 );
```

3.2.1. El problema del bloqueo

El uso de distintas colas de mensajes, asociado al manejo de distintos tipos de tareas o de distintos tipos de mensajes, por parte de un proceso que maneje una semántica de recepción bloqueante puede dar lugar al denominado *problema del bloqueo*.

Suponga que un proceso ha sido diseñado para atender peticiones de varios buzones, nombrados como A y B. En este contexto, el proceso se podría bloquear a la espera de un mensaje en el buzón A. Mientras el proceso se encuentra bloqueado, otro mensaje podría llegar al buzón B. Sin embargo, y debido a que el proceso se encuentra bloqueado en el buzón A, el mensaje almacenado en B no podría ser atendido por el proceso, al menos hasta que se desbloqueara debido a la nueva existencia de un mensaje en A.



¿Cómo es posible atender distintas peticiones (tipos de tareas) de manera eficiente por un conjunto de procesos?

Esta problemática es especialmente relevante en las colas de mensajes POSIX, donde no es posible llevar a cabo una recuperación selectiva por tipo de mensaje².

Existen diversas **soluciones** al problema del bloqueo. Algunas de ellas son específicas del interfaz de paso de mensajes utilizado, mientras que otras, como la que se discutirá a continuación, son generales y se podrían utilizar con cualquier sistema de paso de mensajes.

Dentro del contexto de las situaciones específicas de un sistema de paso de mensajes, es posible utilizar un esquema basado en **recuperación selectiva**, es decir, un esquema que permite que los procesos obtengan mensajes de un buzón en base a un determinado criterio. Por ejemplo, este criterio podría definirse en base al tipo de mensaje, codificado mediante un valor numérico. Otro criterio más exclusivo podría ser el propio identificador del proceso, planteando así un esquema estrechamente relacionado con una comunicación directa.

²En System V IPC, es posible recuperar mensajes de un buzón indicando un tipo.

No obstante, el problema de la recuperación selectiva sólo solventa el problema del bloqueo parcialmente ya que no es posible conocer a priori si un proceso se bloqueará ante la recepción (con semántica bloqueante) de un mensaje asociado a un determinado tipo.

Otra posible opción específica, en este caso incluida en las colas de mensajes de POSIX, consiste en hacer uso de algún tipo de esquema de notificación que sirva para que un proceso sepa cuándo se ha recibido un mensaje en un buzón específico. En el caso particular de POSIX, la operación *mq_notify* permite, previo registro, que un proceso o hilo reciba una señal cuando un mensaje llegue a un determinado buzón.

Desde el punto de vista de las soluciones generales, en la sección 5.18 de [9] se discute una solución al problema del bloqueo denominada **Unified Event Manager** (gestor de eventos unificado). Básicamente, este enfoque propone una biblioteca de funciones que cualquier aplicación puede usar. Así, una aplicación puede registrar un evento, causando que la biblioteca cree un hilo que se bloquea. La aplicación está organizada alrededor de una cola con un único evento. Cuando un evento llega, la aplicación lo desencola, lo procesa y vuelve al estado de espera.

El enfoque general que se plantea en esta sección se denomina **esquema beeper** (ver figura 3.7) y consiste en utilizar un buzón específico que los procesos consultarán para poder recuperar información relevante sobre la próxima tarea a acometer.



El esquema *beeper* planteado en esta sección se asemeja al concepto de localizador o buscador de personas, de manera que el localizador notifica el número de teléfono o el identificador de la persona que está buscando para que la primera se pueda poner en contacto.

Por ejemplo, un proceso que obtenga un mensaje del buzón *beeper* podrá utilizar la información contenido en el mismo para acceder a otro buzón particular. De este modo, los procesos tienen más información para acometer la siguiente tarea o procesar el siguiente mensaje.

En base a este esquema general, los procesos estarán bloqueados por el buzón *beeper* a la hora de recibir un mensaje. Cuando un proceso lo reciba y, por lo tanto, se desbloquee, entonces podrá utilizar el contenido del mensaje para, por ejemplo, acceder a otro buzón que le permita completar una tarea. En realidad, el uso de este esquema basado en dos niveles evita que un proceso se bloquee en un buzón mientras haya tareas por atender en otro.

Así, los procesos estarán típicamente bloqueados mediante la primitiva *receive* en el buzón *beeper*. Cuando un cliente requiera que uno de esos procesos atienda una petición, entonces enviará un mensaje al buzón *beeper* indicando en el contenido el tipo de tarea o el nuevo buzón sobre el que obtener información. Uno de los procesos bloqueados se desbloqueará, consultará el contenido del mensaje y, posteriormen-

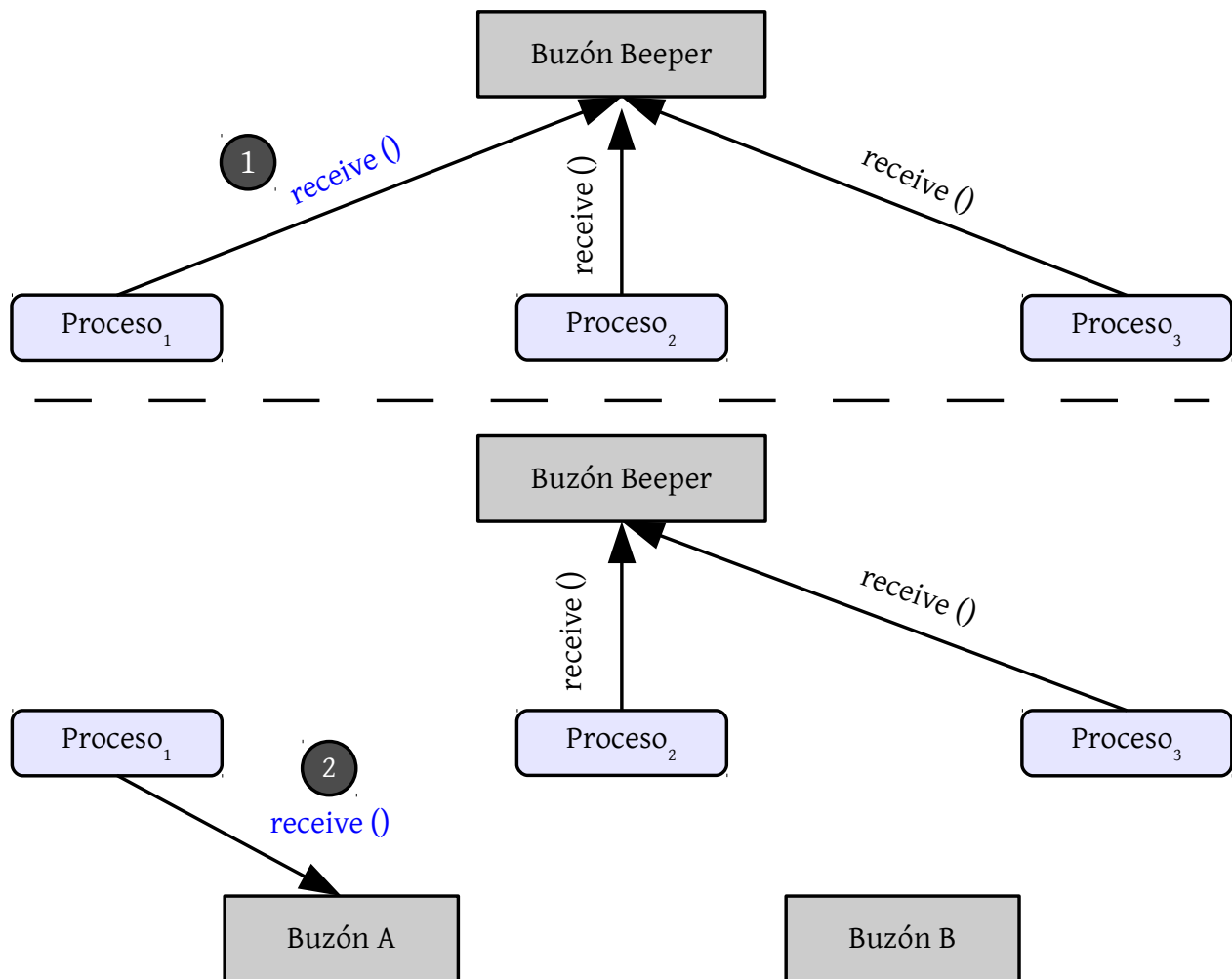


Figura 3.7: Mecanismo de notificación basado en un buzón o cola tipo *beeper* para gestionar el problema del bloqueo.

te, realizará una tarea o accederá al buzón indicado en el contenido del mensaje enviado al *beeper*.

3.3. Problemas clásicos de sincronización

En esta sección se plantean varios problemas clásicos de sincronización, algunos de los cuales ya se han planteado en la sección 2.3, y se discuten cómo podrían solucionarse utilizando el mecanismo del paso de mensajes.

3.3.1. El buffer limitado

El problema del buffer limitado o *problema del productor/consumidor*, introducido en la sección 1.2.1, se puede resolver de una forma muy sencilla utilizando un único buzón o cola de mensajes. Recuerde que en la sección 2.3.1 se planteó una solución a este problema utilizando dos semáforos contadores para controlar, respectivamente, el número de huecos llenos y el número de huecos vacíos en el buffer.

Sin embargo, la solución mediante paso de mensajes es más simple, ya que sólo es necesario utilizar un único buzón para controlar la problemática subyacente. Básicamente, tanto los productores como los consumidores comparten un buzón cuya capacidad está determinada por el número de huecos del buffer.

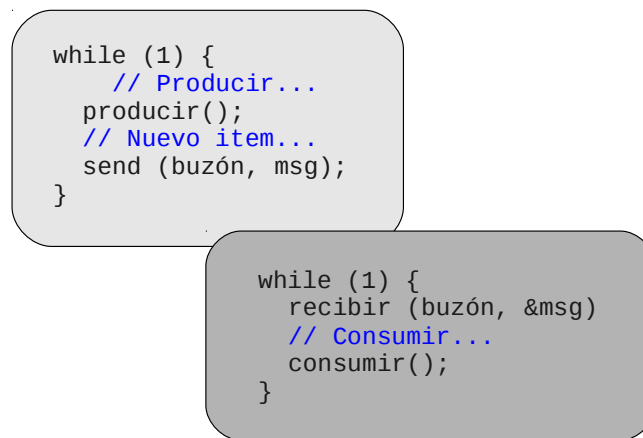


Figura 3.8: Solución al problema del buffer limitado mediante una cola de mensajes.

Inicialmente, el buzón se *rellena* o *inicializa* con tantos mensajes como huecos ocupados haya originalmente, es decir, 0. Tanto los productores como los consumidores utilizan una semántica bloqueante, de manera que

- Si un productor intenta insertar un elemento con el buffer lleno, es decir, si intenta enviar un mensaje al buzón lleno, entonces se quedará bloqueado a la espera de un nuevo hueco (a la espera de que un consumidor obtenga un ítem del buffer y, en consecuencia, un mensaje de la cola).
- Si un consumidor intenta obtener un elemento con el buffer vacío, entonces se bloqueará a la espera de que algún consumidor inserte un nuevo elemento. Esta situación se modela mediante la primitiva de recepción bloqueante.

Básicamente, la cola de mensajes permite modelar el comportamiento de los procesos productor y consumidor mediante las primitivas básicas de envío y recepción bloqueantes.

3.3.2. Los filósofos comensales

El problema de los filósofos comensales, discutido en la sección 2.3.3, también se puede resolver de una manera sencilla utilizando el paso de mensajes y garantizando incluso que no haya un interbloqueo.

Al igual que ocurriría con la solución basada en el uso de semáforos, la solución discutida en esta sección maneja un **array de palillos** para gestionar el acceso exclusivo a los mismos. Sin embargo, ahora cada palillo está representado por una cola de mensajes con un único mensaje que actúa como *token* o testigo entre los filósofos. En otras palabras, para que un filósofo pueda obtener un palillo, tendrá que recibir el mensaje de la cola correspondiente. Si la cola está vacía, entonces se quedará bloqueado hasta que otro filósofo envíe el mensaje, es decir, libere el palillo.

En la figura 3.10 se muestra el **pseudocódigo** de una posible solución a este problema utilizando paso de mensajes. Note cómo se utiliza un array, denominado *palillo*, donde cada elemento de dicho array representa un buzón de mensajes con un único mensaje.

Proceso filósofo

```
while (1) {
    /* Pensar... */
    receive(mesa, &m);
    receive(palillo[i], &m);
    receive(palillo[i+1], &m);
    /* Comer... */
    send(palillo[i], m);
    send(palillo[i+1], m);
    send(mesa, m);
}
```

Figura 3.10: Solución al problema de los filósofos comensales (sin interbloqueo) mediante una cola de mensajes.

Por otra parte, la solución planteada también hace uso de un buzón, denominado *mesa*, que tiene una capacidad de cuatro mensajes. Inicialmente, dicho buzón se rellena con cuatro mensajes que representan el número máximo de filósofos que pueden coger palillos cuando se encuentren en el estado *hambriento*. El objetivo es evitar una situación de interbloqueo que se podría producir si, por ejemplo, todos los filósofos cogen a la vez el palillo que tienen a su izquierda.

Al manejar una semántica bloqueante a la hora de recibir mensajes (intentar adquirir un palillo), la solución planteada garantiza el acceso exclusivo a cada uno de los palillos por parte de un único filósofo. La acción de dejar un palillo sobre la mesa está representada por la primitiva *send*, posibilitando así que otro filósofo, probablemente bloqueado por *receive*, pueda adquirir el palillo para comer.

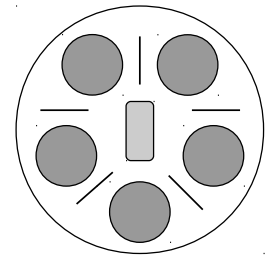


Figura 3.9: Esquema gráfico del problema original de los filósofos comensales (*dining philosophers*).

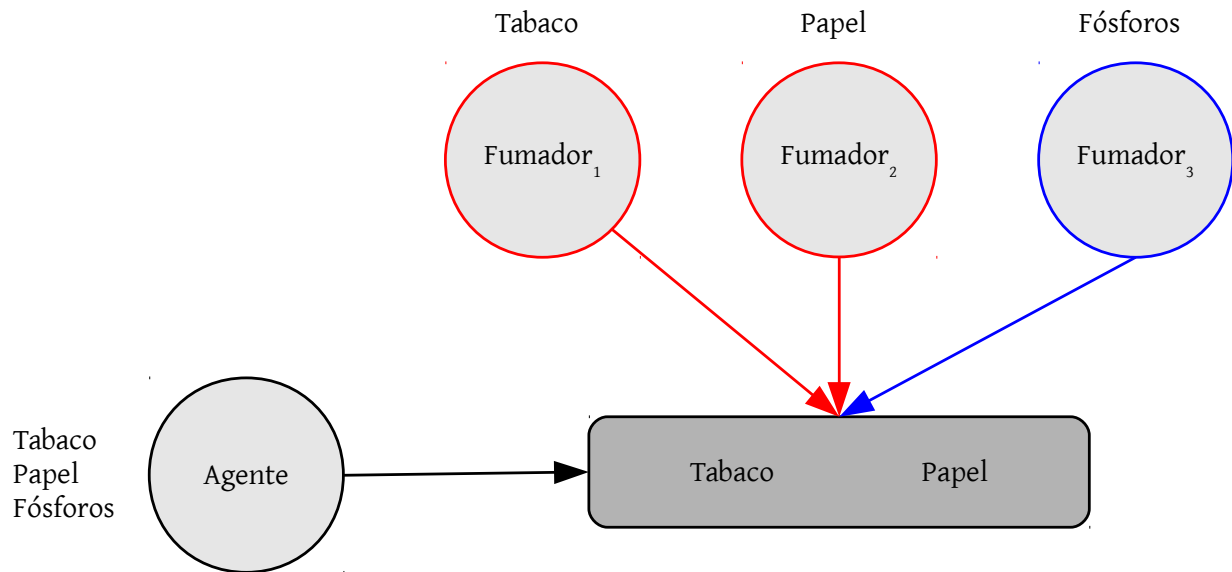


Figura 3.11: Esquema gráfico del problema de los fumadores de cigarrillos.

En el anexo B se muestra una **implementación** al problema de los filósofos comensales utilizando las colas de mensajes POSIX. En dicha implementación, al igual que en la solución planteada en la presente sección, se garantiza que no exista un interbloqueo entre los filósofos.

3.3.3. Los fumadores de cigarrillos

El problema de los fumadores de cigarrillos es otro problema clásico de sincronización. Básicamente, es un problema de **coordinación** entre un agente y tres fumadores en el que intervienen tres ingredientes: i) papel, ii) tabaco y iii) fósforos. Por una parte, el agente dispone de una cantidad ilimitada respecto a los tres ingredientes. Por otra parte, cada fumador dispone de un único elemento, también en cantidad ilimitada, es decir, un fumador dispone de papel, otro de tabaco y otro de fósforos.

Cada cierto tiempo, el agente coloca sobre una mesa, de manera aleatoria, dos de los tres ingredientes necesarios para liarse un cigarrillo (ver figura 3.11). El fumador que tiene el ingrediente restante coge los otros dos de la mesa, se lia el cigarrillo y se lo notifica al agente. Este ciclo se repite indefinidamente.

La problemática principal reside en coordinar adecuadamente al agente y a los fumadores, considerando las siguientes **acciones**:

1. El agente pone dos elementos sobre la mesa.
2. El agente notifica al fumador que tiene el tercer elemento que ya puede liarse un cigarrillo (se simula el acceso del fumador a la mesa).

Proceso agente

```

while (1) {
    /* Genera ingredientes */
    restante = poner_ingredientes();
    /* Notifica al fumador */
    send(buzones[restante], msg);
    /* Espera notificación fumador */
    receive(buzon_agente, &msg);
}

```

Proceso fumador

```

while (1) {
    /* Espera ingredientes */
    receive(mi_buzon, &msg);
    // Liando...
    liar_cigarrillo();
    /* Notifica al agente */
    send(buzon_agente, msg);
}

```

Figura 3.12: Solución en pseudocódigo al problema de los fumadores de cigarrillos usando paso de mensajes.

3. El agente espera a que el fumador le notifique que ya ha terminado de liarse el cigarrillo.
4. El fumador notifica que ya ha terminado de liarse el cigarrillo.
5. El agente vuelve a poner dos nuevos elementos sobre la mesa, repitiéndose así el ciclo.

Este problema se puede resolver utilizando dos **posibles esquemas**:

- Recuperación **selectiva** utilizando un único buzón (ver figura 3.13). En este caso, es necesario integrar en el contenido del mensaje de algún modo el tipo asociado al mismo con el objetivo de que lo recupere el proceso adecuado. Este tipo podría ser A, T, P o F en función del destinatario del mensaje.
- Recuperación **no selectiva** utilizando múltiples buzones (ver figura 3.14). En este caso, es necesario emplear tres buzones para que el agente pueda comunicarse con los tres tipos de agentes y, adicionalmente, otro buzón para que los fumadores puedan indicarle al agente que ya terminaron de liarse un cigarrillo.

Este último caso de recuperación no selectiva está estrechamente ligado al planteamiento de las colas de mensajes POSIX, donde no existe un soporte explícito para llevar a cabo una recuperación selectiva. En la figura 3.12 se muestra el pseudocódigo de una posible solución atendiendo a este segundo esquema.

Como se puede apreciar en la misma, el agente notifica al fumador que completa los tres ingredientes necesarios para fumar a través de un buzón específico, asociado a cada tipo de fumador. Posteriormente, el agente se queda bloqueado (mediante *receive* sobre *buzon_agente*) hasta que el fumador notifique que ya ha terminado de liarse el cigarrillo. En esencia, la sincronización entre el agente y el fumador se realiza utilizando el **patrón rendezvous**.

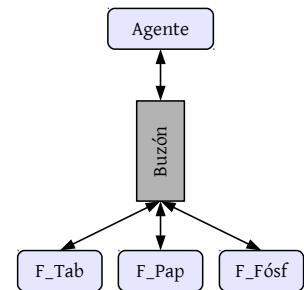


Figura 3.13: Solución al problema de los fumadores de cigarrillos con un único buzón de recuperación selectiva.

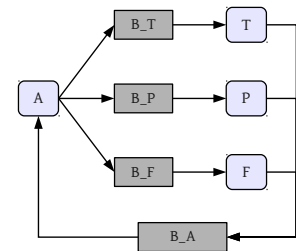


Figura 3.14: Solución al problema de los fumadores de cigarrillos con múltiples buzones sin recuperación selectiva (A=Agente, T=Fumador con tabaco, P=Fumador con papel, F=Fumador con fósforos).

Entero	1	2	...	25	26	27	28	...	51	52	53	54	55	56	57
Traducción	a	b	...	y	z	A	B	...	Y	Z	.	,	!	?	_
Código ASCII	97	98	...	121	122	65	66	...	89	90	46	44	33	63	95

Figura 3.15: Esquema con las correspondencias de traducción. Por ejemplo, el carácter codificado con el valor 1 se corresponde con el código ASCII 97, que representa el carácter 'a'.

3.3.4. Simulación de un sistema de codificación

En esta sección se propone un problema que no se encuadra dentro de los problemas clásicos de sincronización pero que sirve para afianzar el uso del sistema de paso de mensajes para sincronizar procesos y comunicarlos.

Básicamente, se pide construir un sistema que simule a un sencillo sistema de codificación de caracteres por parte de una serie de **procesos de traducción**. En este contexto, existirá un **proceso cliente** encargado de manejar la siguiente información:

- Cadena codificada a descodificar.
- Número total de procesos de traducción.
- Tamaño máximo de subcadena a descodificar, es decir, tamaño de la unidad de trabajo máximo a enviar a los procesos de codificación.

La cadena a descodificar estará formada por una secuencia de números enteros, entre 1 y 57, separados por puntos, que los procesos de traducción tendrán que traducir utilizando el sistema mostrado en la figura 3.15. El tamaño de cada una de las subcadenas de traducción vendrá determinado por el tercer elemento mencionado en el listado anterior, es decir, el tamaño máximo de subcadena a descodificar.

Por otra parte, el sistema contará con un **único proceso de puntuación**, encargado de traducir los símbolos de puntuación correspondientes a los valores enteros comprendidos en el rango 53-57, ambos inclusive. Así, cuando un proceso de traducción encuentre uno de estos valores, entonces tendrá que enviar un mensaje al único proceso de puntuación para que éste lo descodifique y, posteriormente, se lo envíe al cliente mediante un mensaje.

Los procesos de traducción atenderán peticiones hasta que el cliente haya enviado todas las subcadenas asociadas a la cadena de traducción original. En la siguiente figura se muestra un ejemplo concreto con una cadena de entrada y el resultado de su descodificación.

Cadena de entrada	34	15	12	1	55
Cadena descodificada	72	111	108	97	33
Representación textual	'H'	'o'	'l'	'a'	'l'

Figura 3.16: Ejemplo concreto de traducción mediante el sistema de codificación discutido.

La figura 3.17 muestra de manera gráfica el diseño, utilizando colas de mensajes POSIX (sin recuperación selectiva), planteado para solucionar el problema propuesto en esta sección. Básicamente, se utilizan cuatro colas de mensajes en la solución planteada:

- **Buzón de traducción**, utilizado por el proceso cliente para enviar las subcadenas a descodificar. Los procesos de traducción estarán bloqueados mediante una primitiva de recepción sobre dicho buzón a la espera de trabajo.
- **Buzón de puntuación**, utilizado por los procesos de traducción para enviar mensajes que contengan símbolos de puntuación al proceso de puntuación.
- **Buzón de notificación**, utilizado por el proceso de puntuación para enviar la información descodificada al proceso de traducción que solicitó la tarea.
- **Buzón de cliente**, utilizado por los procesos de traducción para enviar mensajes con subcadenas descodificadas.

En la solución planteada se ha aplicado el patrón *rendezvous* en dos ocasiones para sincronizar a los distintos procesos involucrados. En concreto,

- El proceso cliente envía una serie de órdenes a los procesos de traducción, quedando posteriormente a la espera hasta que haya obtenido todos los resultados parciales (mismo número de órdenes enviadas).
- El proceso de traducción envía un mensaje al proceso de puntuación cuando encuentra un símbolo de traducción, quedando a la espera, mediante la primitiva *receive*, de que este último envíe un mensaje con la traducción solicitada.

La figura 3.18 muestra una posible solución en pseudocódigo considerando un sistema de paso de mensajes con recuperación no selectiva.

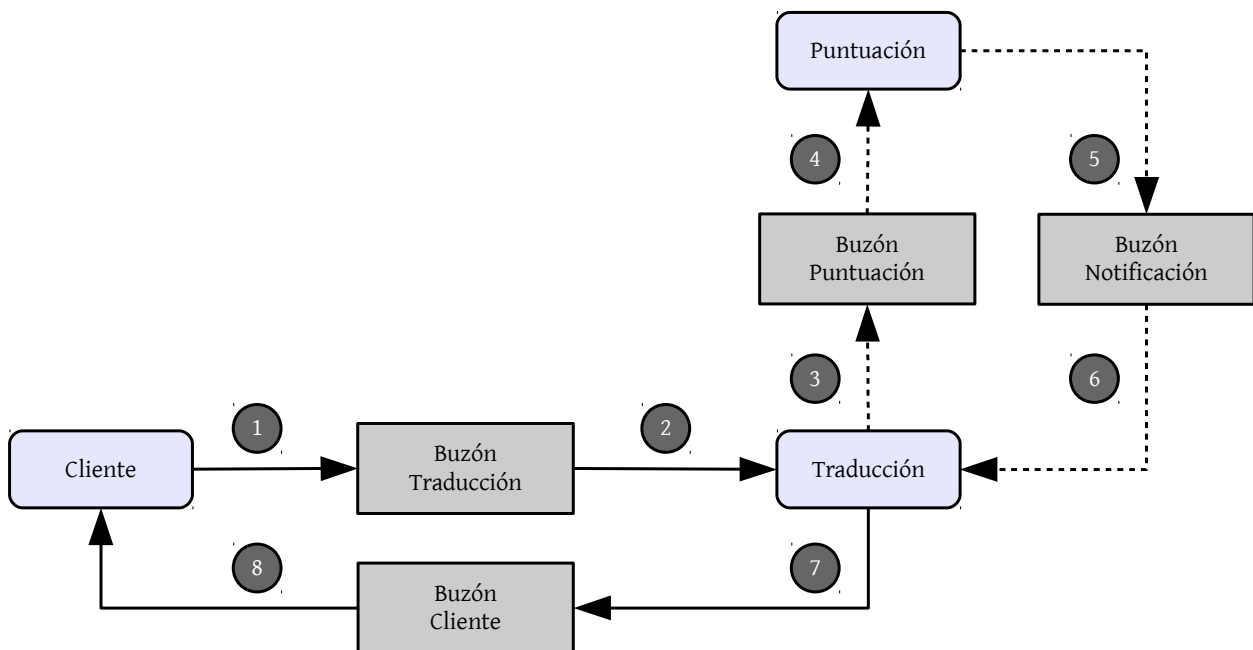


Figura 3.17: Esquema gráfico con la solución planteada, utilizando un sistema de paso de mensajes, para sincronizar y comunicar a los procesos involucrados en la simulación planteada. Las líneas punteadas indican pasos opcionales (traducción de símbolos de puntuación).

Como se puede apreciar, el **proceso cliente** es el responsable de obtener el número de subvectores asociados a la tarea de traducción para, posteriormente, enviar tantos mensajes a los procesos de traducción como tareas haya. Una vez enviados los mensajes al buzón de traducción, el proceso cliente queda a la espera, mediante la primitiva de recepción bloqueante, de los distintos resultados parciales. Finalmente, mostrará el resultado descodificado.

Por otra parte, el **proceso de traducción** permanece en un bucle infinito a la espera de algún trabajo, utilizando para ello la primitiva de recepción bloqueante sobre el buzón de traducción (utilizado por el cliente para enviar trabajos). Si un trabajo contiene un símbolo de traducción, entonces el proceso de traducción solicita ayuda al proceso de puntuación mediante un *rendezvous*, utilizando el buzón de puntuación. Si no es así, entonces traduce el subvector directamente. En cualquier caso, el proceso de traducción envía los datos descodificados al proceso cliente.

Finalmente, el **proceso de puntuación** también permanece en un bucle infinito, bloqueado en el buzón de puntuación, a la espera de alguna tarea. Si la recibe, entonces la procesa y envía el resultado, mediante el buzón de notificación, al proceso de traducción que solicitó dicha tarea.

Proceso cliente

```

obtener_datos(&datos);
crear_buzones();
lanzar_procesos();

calcular_num_subv(datos, &num_sub);

for i in range (0, num_subv) {
    // Generar orden.
    generar_orden(&orden, datos);
    // Solicita traducción.
    send(buzón_traducción, orden);
}

for i in range (0, num_subv) {
    // Espera traducción.
    receive(buzon_cliente, &msg);
    actualizar(datos, msg);
}

mostrar_resultado (datos);

```

Proceso traducción

```

while (1) {
    // Espera tarea.
    receive(buzón_traducción, &orden);
    // Traducción...
    if (es_símbolo(orden.datos)) {
        send(buzón_puntuación, orden);
        receive(buzón_notificación, &msg);
    }
    else
        traducir(&orden, &msg);
    // Notifica traducción.
    send(buzon_cliente, msg);
}

```

Proceso puntuación

```

while (1) {
    receive(buzón_puntuación, &orden);
    traducir(&orden, &msg);
    send(buzon_notificación, msg);
}

```

Figura 3.18: Solución en pseudocódigo que muestra la funcionalidad de cada uno de los procesos del sistema de codificación.

En este problema se puede apreciar la potencia de los sistemas de paso de mensajes, debido a que se pueden utilizar no sólo para sincronizar procesos sino también para compartir información.



4

Capítulo

Otros Mecanismos de Sincronización

En este capítulo se discuten otros mecanismos de sincronización, principalmente de un **mayor nivel de abstracción**, que se pueden utilizar en el ámbito de la programación concurrente. Algunos de estos mecanismos mantienen la misma esencia que los ya estudiados anteriormente, como por ejemplo los semáforos, pero añaden cierta funcionalidad que permite que el desarrollador se abstraiga de determinados aspectos problemáticos.

En concreto, en este capítulo se estudian soluciones vinculadas al uso de lenguajes de programación que proporcionan un soporte nativo de concurrencia, como Ada, y el concepto de monitor como ejemplo representativo de esquema de sincronización de más alto nivel.

Por una parte, el lector podrá asimilar cómo **Ada95** proporciona herramientas y elementos específicos que consideran la ejecución concurrente y la sincronización como aspectos fundamentales, como es el caso de las *tareas* o los *objetos protegidos*. Los ejemplos planteados, como es el caso del problema de los filósofos comensales, permitirán apreciar las diferencias con respecto al uso de semáforos POSIX.

Por otra parte, el **concepto de monitor** permitirá comprender la importancia de soluciones de más alto nivel que solventen algunas de las limitaciones que tienen mecanismos como el uso de semáforos y el paso de mensajes. En este contexto, se estudiará el problema del buffer limitado y cómo el uso de monitores permite plantear una solución más estructurada y natural.

4.1. Motivación

Los mecanismos de sincronización basados en el uso de semáforos y colas de mensajes estudiados en los capítulos 2 y 3, respectivamente, presentan ciertas **limitaciones** que han motivado la creación de otros mecanismos de sincronización que las solventen o las mitiguen. De hecho, hoy en día existen lenguajes de programación con un fuerte soporte nativo de concurrencia y sincronización.

En su mayor parte, estas limitaciones están vinculadas a la propia naturaleza de **bajo nivel de abstracción** asociada a dichos mecanismos. A continuación se discutirá la problemática particular asociada al uso de semáforos y colas de mensajes como herramientas para la sincronización.

En el caso de los **semáforos**, las principales limitaciones son las siguientes:

- Son **propensos a errores** de programación. Por ejemplo, considere la situación en la que el desarrollador olvida liberar un semáforo binario mediante la primitiva *signal*.
- Proporcionan una **baja flexibilidad** debido a su naturaleza de bajo nivel. Recuerde que los semáforos se manejan únicamente mediante las primitivas *wait* y *signal*.

Además, en determinados contextos, los semáforos pueden presentar el problema de la espera activa en función de la implementación subyacente en primitivas como *wait*. Así mismo, su uso suele estar asociado a la manipulación de elementos desde un punto de vista global, presentando una problemática similar a la del uso de variables globales en un programa.

Por otra parte, en el caso de las **colas de mensajes**, las principales limitaciones son las siguientes:

- **No modelan la sincronización de forma natural** para el programador. Por ejemplo, considere el modelado de la exclusión mutua de varios procesos mediante una cola de mensajes con capacidad para un único elemento.
- Al igual que en el caso de los semáforos, proporcionan una **baja flexibilidad** debido a su naturaleza de bajo nivel. Recuerde que las colas de mensajes se basan, esencialmente, en el uso de las primitivas *send* y *receive*.

Este tipo de limitaciones, vinculadas a mecanismos de sincronización de bajo nivel como los semáforos y las colas de mensajes, han motivado la aparición de soluciones de más alto nivel que, desde el punto de vista del programador, se pueden agrupar en dos grandes categorías:

- **Lenguajes de programación** con soporte nativo de concurrencia, como por ejemplo *Ada* o *Modula*.

- **Bibliotecas** externas que proporcionan soluciones, típicamente de más alto nivel que las ya estudiadas, al problema de la concurrencia, como por ejemplo la biblioteca de hilos de ZeroC ICE que se discutirá en la siguiente sección.



Aunque los semáforos y las colas de mensajes pueden presentar ciertas limitaciones que justifiquen el uso de mecanismos de un mayor nivel de abstracción, los primeros pueden ser más adecuados que estos últimos dependiendo del problema a solucionar. Considere las ventajas y desventajas de cada herramienta antes de utilizarla.

En el caso de los lenguajes de programación con un soporte nativo de concurrencia, **Ada** es uno de los casos más relevantes en el ámbito comercial, ya que originalmente fue diseñado¹ para construir sistemas de tiempo real críticos prestando especial importancia a la fiabilidad. En este contexto, Ada proporciona elementos específicos que facilitan la implementación de soluciones con necesidades de concurrencia, sincronización y tiempo real.

En el caso de las bibliotecas externas, uno de sus principales objetivos de diseño consiste en solventar las limitaciones previamente introducidas. Para ello, es posible incorporar nuevos mecanismos de sincronización de más alto nivel, como por ejemplo los monitores, o incluso añadir más capas software que permitan que el desarrollador se abstraiga de aspectos específicos de los mecanismos de más bajo nivel, como por ejemplo los semáforos.

4.2. Concurrency en Ada 95

Ada es un lenguaje de programación orientado a objetos, fuertemente tipado, multi-propósito y con un gran soporte para la programación concurrente. Ada fue diseñado e implementado por el Departamento de Defensa de los Estados Unidos. Su nombre se debe a *Ada Lovelace*, considerada como la primera programadora en la historia de la Informática.

Uno de los principales pilares de Ada reside en una filosofía basada en la reducción de errores por parte de los programadores tanto como sea posible. Dicha filosofía se traslada directamente a las construcciones del propio lenguaje. Esta filosofía tiene como objetivo principal la creación de programas altamente legibles que conduzcan a una maximización de su **mantenibilidad**.

Ada se utiliza particularmente en aplicaciones vinculadas con la seguridad y la fiabilidad, como por ejemplos las que se enumeran a continuación:

¹Ada fue diseñado por el Departamento de Defensa de EEUU.

- Aplicaciones de defensa (e.g. DoD de EEUU).
- Aplicaciones de aeronáutica (e.g. empresas como *Boeing* o *Airbus*).
- Aplicaciones de gestión del tráfico aéreo (e.g. empresas como *In-dra*).
- Aplicaciones vinculadas a la industria aeroespacial (e.g. NASA).

Resulta importante resaltar que en esta sección se realizará una discusión de los elementos de **sopORTE para la concurrencia y la sincronización** ofrecidos por Ada 95, así como las estructuras propias del lenguaje para las necesidades de tiempo real. Sin embargo, no se llevará a cabo un recorrido por los elementos básicos de este lenguaje de programación.

En otras palabras, en esta sección se considerarán especialmente los elementos básicos ofrecidos por Ada 95 para la programación concurrente y los sistemas de tiempo real. Para ello, la perspectiva abordada girará en torno a dos conceptos fundamentales:

1. El uso de **esquemas de programación concurrente de alto nivel**, como por ejemplo los objetos protegidos estudiados en la sección 4.2.2.
2. El uso práctico de un lenguaje de programación orientado al **diseño y desarrollo de sistemas concurrentes**.

Antes de pasar a discutir un elemento esencial de Ada 95, como es el concepto de tarea, el siguiente listado de código muestra una posible implementación del clásico programa *¡Hola Mundo!*

Listado 4.1: *¡Hola Mundo!* en Ada 95.

```
1 with Text_IO; use Text_IO;  
2  
3 procedure HolaMundo is  
4 begin  
5   Put_Line("Hola Mundo!");  
6 end HolaMundo;
```

Para poder compilar y ejecutar los ejemplos implementados en Ada 95 se hará uso de la herramienta *gnatmake*, la cual se puede instalar en sistemas GNU/Linux mediante el siguiente comando:

```
$ sudo apt-get install gnat
```



GNAT es un compilador basado en la infraestructura de *gcc* y utilizado para compilar programas escritos en Ada.

Una vez instalado GNAT, la compilación y ejecución de programas escritos en Ada es trivial. Por ejemplo, en el caso del ejemplo *¡Hola Mundo!* es necesario ejecutar la siguiente instrucción:

```
$ gnatmake holamundo.adb
```

4.2.1. Tareas y sincronización básica

En Ada 95, una tarea representa la **unidad básica de programación concurrente** [3]. Desde un punto de vista conceptual, una tarea es similar a un proceso. El siguiente listado de código muestra la definición de una tarea sencilla.

Listado 4.2: Definición de tarea simple en Ada 95.

```
1 -- Especificación de la tarea T.
2 task type T;
3
4 -- Cuerpo de la tarea T.
5 task body T is
6   -- Declaraciones locales al cuerpo.
7   -- Orientado a objetos (Duration).
8   periodo:Duration := Duration(10);
9   -- Identificador de tarea.
10  id_tarea:Task_Id := Null_Task_Id;
11
12 begin
13   -- Current Task es similar a getpid().
14   id_tarea := Current_Task;
15   -- Bucle infinito.
16   loop
17     -- Retraso de 'periodo' segundos.
18     delay periodo;
19     -- Impresión por salida estándar.
20     Put("Identificador de tarea: ");
21     Put_Line(Image(ID_T));
22   end loop;
23
24 end T;
```

Como se puede apreciar, la tarea está compuesta por dos elementos distintos:

1. La especificación (declaración) de la tarea T.
2. El propio cuerpo (definición) de la tarea T.

El **lanzamiento de tareas** es sencillo ya que éstas se activan de manera automática cuando la ejecución llega al procedimiento (*procedure*) en el que estén declaradas. Así, el procedimiento *espera* la finalización de dichas tareas. En el siguiente listado de código se muestra un ejemplo.

Es importante resaltar que *Tarea1* y *Tarea2* se ejecutarán concurrentemente después de la sentencia *begin* (línea 14), del mismo modo que ocurre cuando se lanzan diversos procesos haciendo uso de las primitivas que proporciona, por ejemplo, el estándar POSIX.

Listado 4.3: Activación de tareas en Ada 95.

```
1 -- Inclusión de paquetes de E/S e
2 -- identificación de tareas.
3 with Ada.Text_IO; use Ada.Text_IO;
4 with Ada.Task_Identification;
5 use Ada.Task_Identification;
6
7 procedure Prueba_Tareas is
8   -- Inclusión del anterior listado de código...
9
10  -- Variables de tipo T.
11    Tarea1:T;
12    Tarea2:T;
13
14  begin
15    -- No se hace nada, pero las tareas se activan.
16    null;
17  end Prueba_Tareas;
```

La representación y el lanzamiento de tareas se completa con su **finalización**. Así, en Ada 95, una tarea finalizará si se satisface alguna de las siguientes situaciones [3]:

1. La ejecución de la tarea se completa, ya sea de manera normal o debido a la generación de alguna excepción.
2. La tarea ejecuta una alternativa de finalización mediante una instrucción *select*, como se discutirá con posterioridad.
3. La tarea es abortada.

La **sincronización básica** entre tareas en Ada 95 se puede realizar mediante la definición de *entries* (entradas). Este mecanismo de interacción proporcionado por Ada 95 sirve como mecanismo de comunicación y de sincronización entre tareas. Desde un punto de vista conceptual, un *entry* se puede entender como una llamada a un procedimiento remoto de manera que

1. la tarea que invoca suministra los datos a la tarea receptora,
2. la tarea receptora ejecuta el *entry*,
3. la tarea que invoca recibe los datos resultantes de la invocación.

De este modo, un *entry* se define en base a un identificador y una serie de parámetros que pueden ser de entrada, de salida o de entrada/salida. Esta última opción, a efectos prácticos, representa un paso de parámetros por referencia.

En el contexto de la sincronización mediante *entries*, es importante hacer especial hincapié en que el *entry* lo ejecuta la tarea invocada, mientras que la que invoca se queda bloqueada hasta que la ejecución del *entry* se completa. Este esquema de sincronización, ya estudiado en la sección 2.3.5, se denomina sincronización mediante **Rendez-vous**.

Antes de estudiar cómo se puede modelar un problema clásico de sincronización mediante el uso de *entries*, es importante considerar lo que ocurre si, en un momento determinado, una tarea no puede atender una invocación sobre un *entry*. En este caso particular, la tarea que invoca se bloquea en una cola FIFO vinculada al propio *entry*, del mismo modo que suele plantearse en la implementación de los semáforos y la primitiva *wait*.

Modelando el problema del buffer limitado

El problema del buffer limitado, también conocido como problema del productor/consumidor, plantea la problemática del acceso concurrente al mismo por parte de una serie de productores y consumidores de información. En la sección 1.2.1 se describe esta problemática con mayor profundidad, mientras que la sección 2.3.1 discute una posible implementación que hace uso de semáforos.

En esta sección, tanto el propio *buffer* en el que se almacenarán los datos como los procesos *productor* y *consumidor* se modelarán mediante tareas. Por otra parte, la gestión del acceso concurrente al buffer, así como la inserción y extracción de datos, se realizará mediante dos *entries*, denominados respectivamente *escribir* y *leer*.

Debido a la necesidad de proporcionar distintos servicios, Ada 95 proporciona la instrucción ***select*** con el objetivo de que una tarea servidora pueda atender múltiples *entries*. Este esquema proporciona un mayor nivel de abstracción desde el punto de vista de la tarea que solicita o invoca un servicio, ya que el propio entorno de ejecución garantiza que dos *entries*, gestionadas por una sentencia *select*, nunca se ejecutarán de manera simultánea.

El siguiente listado de código muestra una posible implementación de la tarea **Buffer** para llevar a cabo la gestión de un único elemento de información.

Como se puede apreciar, el acceso al contenido del buffer se realiza mediante *Escribir* y *Leer*. Note cómo el parámetro de este último *entry* es de salida, ya que *Leer* es la operación de extracción de datos.

Listado 4.4: Implementación de la tarea Buffer en Ada 95.

```

1  -- Buffer de un único elemento.
2  task Buffer is
3      entry Escribir (Dato: Integer);
4      entry Leer (Dato: out Integer);
5  end Buffer;
6
7  task body Buffer is
8      Temp: Integer;
9  begin
10     loop
11         select
12             -- Almacenamiento del valor entero.
13             accept Escribir (Dato: Integer) do
14                 Temp := Dato;
15             end Escribir;
16         or
17             -- Recuperación del valor entero.
18             accept Leer (Dato: out Integer) do
19                 Dato := Temp;
20             end Leer;
21         end select;
22     end loop;
23 end Buffer;

```

A continuación se muestra una posible implementación, a modo de ejemplo, de la tarea **Consumidor**.

Listado 4.5: Implementación de la tarea Consumidor en Ada 95.

```

1  task Consumidor;
2
3  task body Consumidor is
4      Dato : Integer;
5
6  begin
7      loop
8          -- Lectura en 'Dato'.
9          Buffer.Leer(Dato);
10         Put_Line("[Consumidor] Dato obtenido...");
11         Put_Line(Dato'img);
12         delay 1.0;
13     end loop;
14
15 end Consumer;

```

El buffer de la versión anterior sólo permitía gestionar un elemento que fuera accedido de manera concurrente por distintos consumidores o productores. Para manejar realmente un buffer limitado, es decir, un buffer con una capacidad máxima de N elementos, es necesario modificar la tarea *Buffer* añadiendo un array de elementos, tal y como muestra el siguiente listado de código.

Como se puede apreciar, el buffer incluye lógica adicional para controlar que no se puedan insertar nuevos elementos si el buffer está lleno o, por el contrario, que no se puedan extraer elementos si el buffer está vacío. Estas dos situaciones se controlan por medio de las denominadas **barreras** o **guardas**, utilizando para ello la palabra reservada *when* en Ada 95. Si la condición asociada a la barrera no se